



DEPARTMENT OF
COMPUTER SCIENCE

JOÃO BERNARDO GUERREIRO DOS SANTOS

BSc in Computer Science and Engineering

FINITE-STATE TRANSDUCERS IN OCAMLFLAT/OFLAT

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

March, 2026

FINITE-STATE TRANSDUCERS IN OCAMLFLAT/OFLAT

JOÃO BERNARDO GUERREIRO DOS SANTOS

BSc in Computer Science and Engineering

Supervisor: Artur Miguel Dias
Assistant Professor, NOVA University Lisbon

Examination Committee

Chair: Nuno Manuel Ribeiro Preguiça
Full Professor, NOVA School of Science and technology

Examiner: Simão Melo de Sousa
Full Professor, University of Algarve

Supervisor: Artur Miguel Dias
Assistant Professor, NOVA School of Science and technology

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
March, 2026

Finite-state transducers in OCamlFLAT/OFLAT

Copyright © João Bernardo Guerreiro dos Santos, NOVA School of Science and Technology, NOVA University Lisbon.

The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

This document was produced with the lua $\LaTeX$ [1] typesetting system and the [NOVAthesis](https://github.com/joaomloureiro/novathesis) template (v7.10.7) [2].

- [1] The $\LaTeX$ Project team. *$\LaTeX$ – A document preparation system*. 1985. URL: <https://www.latex-project.org>.
- [2] J. M. Lourenço. *The NOVAthesis $\LaTeX$ Template User's Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomloureiro/novathesis/raw/main/template.pdf>.

ACKNOWLEDGEMENTS

First, I want to thank my advisor Artur Miguel Dias for all the help and counseling given throughout the elaboration of this dissertation and the engaging discussions we had. I would also like to thank my friends for having my back during all these years. Especially during the pandemic. Finally, thank you to my family who always supported me and were patient with me during this journey. I would not be here without any of them.

I would also like to acknowledge João M. Lourenço for developing and maintaining the NOVAtesis $\text{\LaTeX}$ template used to prepare this dissertation [10].

ABSTRACT

This dissertation presents the design and implementation of finite state transducers (FSTs) within the OCamlFLAT/OFLAT application, a web-based educational tool for exploring concepts in computation theory developed at FCT NOVA. Finite state transducers extend finite automata by associating output symbols with transitions, enabling the modeling of transformations from input to output strings.

The current system consists of two separate components: Firstly, we have the OCamlFLAT library, a library with all the data structures and logic regarding the theoretical concepts of its supported models. And secondly, the OFLAT web application, which is built on top of the OCamlFLAT library, provides a pedagogical interactive graphic interface for users to explore the models supported by the library. Both tools were written in OCaml, deliberately aiming to use a functional declarative style, which in some cases manages to closely resemble the theoretical formalisms taught to students. It is a system that is in constant evolution, with new features being added to both the library and the web application.

The work to be developed in this thesis involves the extension of these tools with new operations over transducers, such as translation, acceptance, generation, classification and conversion. Additionally, the new concepts must be integrated in OFLAT's ecosystem, which includes model composition, and pedagogical exercises for students to practice and test their knowledge.

The resulting system will allow users to build, modify, and analyze FSTs visually, supporting better comprehension of theoretical principles through experimentation.

RESUMO

Esta dissertação apresenta o projeto e a implementação de transdutores finitos (FSTs) na aplicação OCamlFLAT/OFLAT, uma ferramenta educacional baseada na web para a exploração de conceitos de teoria da computação, desenvolvida na FCT NOVA. Os transdutores finitos estendem os autómatos finitos ao associar símbolos de saída às transições, permitindo a modelação de transformações entre cadeias de entrada e de saída.

O sistema atual é composto por dois componentes distintos: a biblioteca OCamlFLAT, que contém todas as estruturas de dados e lógica associadas aos conceitos teóricos dos modelos suportados; e a aplicação web OFLAT, construída sobre a biblioteca OCamlFLAT, que fornece uma interface gráfica interativa com fins pedagógicos para que os utilizadores possam explorar os modelos suportados pela biblioteca. Ambas as ferramentas foram desenvolvidas em OCaml, procurando adotar deliberadamente um estilo funcional declarativo, que em alguns casos se aproxima dos formalismos teóricos ensinados aos alunos. Trata-se de um sistema em constante evolução, com novas funcionalidades a serem acrescentadas continuamente tanto à biblioteca como à aplicação web.

O trabalho a desenvolver nesta dissertação envolve a extensão destas ferramentas com novas operações sobre transdutores, tais como tradução, aceitação, geração, classificação e conversão. Adicionalmente, os novos conceitos devem ser integrados no ecossistema do OFLAT, o que inclui a composição de modelos e a criação de exercícios pedagógicos para os alunos praticarem e testarem os seus conhecimentos.

O sistema resultante permitirá aos utilizadores construir, modificar e analisar transdutores finitos de forma visual, promovendo uma melhor compreensão dos princípios teóricos através da experimentação.

DISCLOSURE OF USAGE OF GENERATIVE ARTIFICIAL INTELLIGENCE

The author, João Bernardo Guerreiro dos Santos, acknowledges the use of the following generative artificial intelligence tools in the development of this thesis. These technologies were utilized under strict supervision, and all AI-generated outputs were critically reviewed and verified for accuracy. The author accepts full responsibility for the validity and originality of the content, confirming that the use of these tools aligns with all institutional policies and standards of academic integrity.¹

Generative AI tools used: ChatGPT-5.5 and Gemini 3.1 Pro.

Activities supported by Generative Artificial Intelligence Tools:²

Conceptualization

- | | | |
|--|--|--|
| ☐ Idea generation | ☐ Formulating research questions and hypotheses | ☐ Hypothesis viability assessment |
| ☐ Defining the research objective | ☐ Feasibility assessment and risk evaluation | ☐ Simulated debate and argument testing |

Literature Review

- | | | |
|--|--|--|
| ☒ Search and Discovery | ☐ Concept Mapping and Systematization | ☐ Gap Identification and Novelty Evaluation |
| ☒ Literature summarization and synthesis | ☐ Market/patent landscape analysis | ☒ Cross-lingual literature comprehension |

Methodology

- | | | |
|---|--|--|
| ☐ Experimental Design Optimization | ☐ Development of experimental or research protocols | ☐ Methodological Instrument Selection |
|---|--|--|

¹This declaration was generated using Lua $\text{\LaTeX}$ [23] and the $\text{\LaTeX}$ package aidisclose [9].

²This list extends the GAIDeT taxonomy [27].

Software Development and Automation

- | | | |
|--|--|---|
| ☐ Code Generation | ☒ Debugging and Repair | ☐ Algorithm design |
| ☒ Refactoring and Optimization | ☐ Process automation | ☒ Code documentation and comment generation |

Data Management

- | | | |
|---|--|--|
| ☐ Data collection | ☐ Data analysis | ☐ Synthetic data generation |
| ☐ Validation | ☐ Quantitative Plotting and Charting | ☐ De-identification and anonymization support |
| ☐ Data cleaning | ☒ Reproducibility and rerun checks | ☐ Audio-to-text transcription and diarization |
| ☐ Data curation and organization | ☐ Data labeling and annotation assistance | |

Visuals and Multimedia

- | | | |
|---|--|--|
| ☐ Synthetic Asset Generation | ☐ Image Enhancement and Editing | ☐ Diagrammatic and Schematic Design |
|---|--|--|

Writing and Editing

- | | | |
|--|---|---|
| ☒ Drafting Text | ☐ Formulation of conclusions | ☒ Citation formatting and bibliography management |
| ☐ Polishing and Editing | ☐ Tone Adjustment | ☐ Press releases and outreach materials |
| ☐ Abstract and Executive Summary Generation | ☒ Translation | ☐ Title Generation |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Quality Assurance

- | | | |
|--|---|--|
| ☐ Simulated Peer Review | ☐ Identification of limitations | ☐ Publication support |
| ☐ Consistency checking against field trends | ☐ Publication strategy and journal selection | |

CONTENTS

Abstract	iv
Resumo	v
Disclosure of Usage of Generative Artificial Intelligence	vi
Contents	viii
List of Figures	xi
List of Tables	xiii
1 Introduction	1
1.1 Context	1
1.2 Objectives	1
1.3 Contributions	2
1.4 Structure of the Document	2
2 Background	4
2.1 Introduction	4
2.1.1 Formal Language and Automata Theory	4
2.2 Languages and Grammars	5
2.2.1 Chomsky Language Hierarchy	5
2.2.2 Regular Languages	5
2.2.3 Regular Expressions	6
2.3 Finite Automata	7
2.3.1 Deterministic Finite Automata (DFAs)	7
2.3.2 Nondeterministic Finite Automata (NFAs)	8
2.4 Finite-State Transducers	10
2.4.1 Operations on Finite-State Transducers	10
2.4.2 Mealy Machines	11
2.4.3 Moore Machines	12
2.4.4 Moore and Mealy Machine Equivalence	13
2.4.5 Mealy Machine Minimization	14

2.4.6	Moore Machine Minimization	15
2.4.7	General Finite-State Transducers	16
2.4.8	Relevance to OCamlFLAT and OFLAT	18
2.5	Conclusion	18
3	Related Work	19
3.1	Introduction	19
3.2	JFLAP: Java Formal Languages and Automata Package	19
3.3	FAdo	21
3.4	Automaton Simulator	21
3.5	Flap.js	23
3.6	Other tools	24
3.7	Comparison to OFLAT	24
4	OCamlFLAT / OFLAT	26
4.1	Technology Stack	26
4.2	OCamlFLAT Library	26
4.3	OFLAT	27
4.3.1	User Interface Layout	28
4.3.2	OFLAT Architecture	29
4.3.3	Current Limitations	30
4.4	Conclusion	30
5	Technologies	31
5.1	Why OCaml for OFLAT	31
5.2	OCaml Programming Language	31
5.2.1	Declarative Programming	31
5.3	Interoperability using js_of_ocaml	32
5.3.1	Bindings	32
5.4	Cytoscape.js for Graph Visualization	34
5.5	Web Technologies Used in OFLAT	34
6	OCamlFLAT Library Implementation	35
6.1	Transducer Representation	35
6.2	Validation and Properties	36
6.3	Execution Engine	39
6.4	Algorithms	41
6.4.1	Determinization	41
6.4.2	Minimization	41
6.4.3	Conversion to Turing Machine	42
6.5	Composition and Operations	42
6.6	Integration Points Exposed to OFLAT	44

7	OFLAT Design and Features	45
7.1	Design Goals	45
7.2	Workspace and Graph Interaction	45
7.3	Information Box, Poppers, and Context Menus	46
7.4	Step-by-Step Execution Experience	47
7.5	Trace Tables and Minimization Views	47
7.6	Supported Models and Scope	48
7.7	FST-Specific Workflows	48
7.8	Navigation to the Implementation View	49
8	OFLAT Implementation	50
8.1	Workspace and Graph Interaction	50
8.2	Information Box, Poppers, and Menus	51
8.3	Step-by-Step Execution Pipeline	52
8.4	Tables, Traces, and Minimization	53
8.5	Exercise Handling	53
8.6	Integration Boundaries	54
9	Evaluation and Validation	55
9.1	Testing Methodology	55
9.2	Use Cases	56
9.2.1	Case 1: Simulating a Moore Machine	56
9.2.2	Case 2: Non-Deterministic FST Execution	56
9.2.3	Case 3: FST Minimization	57
9.2.4	Case 4: Determinization Failure (Conflicting Outputs)	57
9.2.5	Case 5: Output-Aware Exercises	58
9.2.6	Case 6: Scale (Branching Explosion)	59
9.3	Discussion and Limitations	60
10	Conclusion and Future Work	61
10.1	Conclusion	61
10.2	Summary of Contributions	61
10.3	Future Work	62
	Bibliography	63

LIST OF FIGURES

2.1	Example of a basic DFA.	8
2.2	Example of a basic NFA.	9
2.3	Example of a basic mealy machine.	12
2.4	Example of a basic Moore machine.	13
2.5	Example of a Mealy machine and its equivalent Moore machine.	14
2.6	Example of a mealy machine and its minimal equivalent.	14
2.7	Example of a non-deterministic FST that cannot be determinized.	17
3.1	Model list available in JFLAP.	20
3.2	A simple Mealy machine in JFLAP.	20
3.3	Left panel of the Automaton Simulator.	22
3.4	Central area of the Automaton Simulator.	23
3.5	Flap.js user interface.	24
4.1	OFLAT Side Menu	28
4.2	OFLAT Workspace showing a finite automaton	29
7.1	OFLAT canvas with input:output edge labels.	46
7.2	Information box and popper displaying state/input/output and configuration counts.	47
7.3	Step-by-step execution with highlighted path, input cursor, and accumulated output.	47
7.4	Trace table for an FST run showing state, remaining input, and produced output.	48
8.1	Rendering view with input:output edges and minimized coloring.	51
8.2	Exercise feedback: input→output cases with actual outputs and property results.	54
9.1	Moore machine execution highlighting state-based outputs and trace table.	56
9.2	Nondeterministic FST execution with branch highlights, popper counts, and output tracking.	57

9.3	FST minimization view with equivalence-class coloring and preserved input:output labels.	57
9.4	Determinization failure due to epsilon transitions; error surfaced while graph is preserved.	58
9.5	Determinization failure due to conflicting outputs; error surfaced while graph is preserved.	58
9.6	Output-aware exercise feedback with arrow notation, highlighting mismatches.	59
9.7	Scale test with high branching; UI remains responsive while showing many active configurations.	60

LIST OF TABLES

3.1 Positioning OFLAT relative to related educational tools.	25
--	----

LIST OF LISTINGS

INTRODUCTION

1.1 Context

Theory of Computation is a fundamental area in computer science that includes various fields of study, including this work's focus: Formal Languages and Automata Theory (FLAT). These play a critical role in modeling computational processes, verifying system behavior, and designing compilers and interpreters.

Despite its theoretical significance, FLAT concepts can be difficult to grasp due to their abstract nature. For students, interactive visualization and step-by-step simulation are critical aids. OCamlFLAT/OFLAT, developed at NOVA School of Science and Technology, already supports finite automata, grammars, pushdown automata, and Turing machines. However, it lacks finite-state transducers (FST), a model that adds output generation to automata and is central to translation, lexical analysis, and many teaching examples.

While FSTs are well-established in the literature, pedagogical tools that let learners build, simulate, and analyze them with output tracking are scarce or incomplete. This dissertation fills that gap by extending both the OCamlFLAT library and the OFLAT web interface with native FST support, aiming to deliver a consistent, interactive learning experience alongside the existing models.

1.2 Objectives

The primary goal of this thesis is to design and implement support for finite-state transducers within the OCamlFLAT/OFLAT ecosystem. This includes:

- Defining data structures and algorithms to model FSTs (including acceptance with expected outputs, determinism checks, and minimization).
- Exposing output-aware operations in OCamlFLAT (e.g., `acceptOut`, `acceptExpectedOutput`, `inputEdges`) consistent with existing automata APIs.
- Integrating FST editing and step-by-step execution into OFLAT with input:output edge labels, accumulated-output display, and nondeterministic branch visuals.

- Enabling exercise checking with input->output examples so UI feedback matches library semantics.

By enriching the system with transducer functionality, we aim to provide a more comprehensive pedagogical platform that supports a broader spectrum of the Chomsky hierarchy and theoretical computation models. This will enhance both teaching and learning by enabling students and instructors to explore complex language transformations through hands-on experimentation.

1.3 Contributions

The main contributions of this dissertation are:

- A finite-state transducer extension to OCamlFLAT with output-aware acceptance, determinism checks, and minimization utilities reused by the UI.
- An OFLAT integration that supports interactive FST construction, step-by-step execution with accumulated outputs, branch highlighting, popper counts, and exercise checking for input->output cases.
- An evaluation covering unit tests, integration checks in the browser, and representative FST use cases, with guidance for future improvements.

1.4 Structure of the Document

The remainder of this document is structured as follows.

- Chapter 2 introduces the necessary concepts in formal languages and automata, focusing on finite-state transducers.
- Chapter 3 surveys related tools and positions this work against existing automata/FST teaching environments.
- Chapter 4 presents the OCamlFLAT/OFLAT platform as it existed before this work.
- Chapter 5 outlines the key technologies used (OCaml, `js_of_ocaml`, Cytoscape.js, and supporting libraries).
- Chapter 6 details the OCamlFLAT FST implementation (data types, acceptance, determinism, minimization).
- Chapter 7 describes the OFLAT design and user experience for FSTs (interaction model, visuals, feedback surfaces).
- Chapter 8 explains the OFLAT implementation, including event wiring, data flow, rendering, and exercise checking.

- Chapter 9 presents the evaluation methodology, test layers, and representative use cases.
- Chapter 10 summarizes the results and outlines future work.

BACKGROUND

2.1 Introduction

The OFLAT application builds on models from the Theory of Computation. To keep the rest of the document easy to follow, we first recap the core background that motivates OFLAT: languages, grammars, automata, and finally Finite-State Transducers (FSTs), which are the central model used in the implementation.

2.1.1 Formal Language and Automata Theory

Formal Language and Automata Theory (referred to as FLAT from here onwards), is technically a composition of two subjects, Formal Language Theory and Automata Theory that, while they are different subjects, they are related and often studied and mentioned together. Automata theory is the study of abstract machines and automata, as well as the computational problems that can be solved using them. An automaton (automata in plural) is an abstract self-propelled computing device which follows a predetermined sequence of operations automatically. An automaton with a finite number of states is called a finite automaton (FA) or finite-state machine (FSM).

A formal language is a set of strings whose symbols are taken from a set called "alphabet". The alphabet of a formal language consists of symbols that concatenate into strings (also called "words"). A formal language is often defined by means of a formal grammar such as a regular grammar or context-free grammar. In computer science, formal languages are used, among others, as the basis for defining the grammar of programming languages and formalized versions of subsets of natural languages, in which the words of the language represent concepts that are associated with meanings or semantics.

In FLAT, automata are used as finite representations of formal languages that may be infinite. Automata are often classified by the class of formal languages they can recognize, as in the Chomsky hierarchy (detailed later), which describes a nesting relationship between major classes of automata.

2.2 Languages and Grammars

Languages can be expressed over two different methods: Recognition (via the use of automata) and Generation (via the use of grammars). In generation a language is represented by a grammar and is a set of all the words that can be derived from the start symbol of the grammar. While in recognition, a language is represented by an acceptor (usually an automaton) and is a set of all the words that are accepted by the automaton. Finite automata and FSTs are recognizers so our focus will be on them, although with FSTs there is more nuance to that concept, as we will see later.

2.2.1 Chomsky Language Hierarchy

The Chomsky hierarchy[24] is a containment hierarchy of classes of formal grammars. Noam Chomsky theorized that four different classes of formal grammars existed that could generate increasingly complex languages. Each class can also completely generate the language of all inferior classes (set inclusive). The general idea of a hierarchy of grammars was first described by Noam Chomsky in "Three models for the description of language". Marcel-Paul Schützenberger also played a role in the development of the theory of formal languages; the paper "The algebraic theory of context free languages" describes the modern hierarchy, including context-free grammars. Independently, alongside linguists, mathematicians were developing models of computation (via automata). Parsing a sentence in a language is similar to computation, and the grammars described by Chomsky proved to both resemble and be equivalent in computational power to various machine models. The Chomsky hierarchy is often represented as follows:

Type 0: Recursively enumerable languages, generated by unrestricted grammars and recognized by Turing machines.

Type 1: Context-sensitive languages, generated by context-sensitive grammars and recognized by linear bounded automata.

Type 2: Context-free languages, generated by context-free grammars and recognized by pushdown automata. Common in programming languages and syntactic structures.

Type 3: Regular languages, the simplest class, generated by regular grammars and recognized by finite automata. They represent patterns describable by regular expressions.

Finite-State Transducers operate at the Type 3 level, dealing with regular languages.

2.2.2 Regular Languages

Regular languages are a class of formal languages that can be defined by a regular expression and recognized by finite automata. They represent the simplest type of language in the Chomsky hierarchy (Type 3). A language L is said to be regular if there exists a finite automaton that accepts exactly the strings in L .

Regular languages are defined semantically over an alphabet Σ as follows:

- The empty language $\emptyset$ is a regular language.

- For each $a \in \Sigma$ (a belongs to Σ), the singleton language a is a regular language.
- If A is a regular language, A^* (Kleene star) is a regular language. Due to this, the empty string language ε is also regular.
- If A and B are regular languages, then $A \cup B$ (union) and AB (concatenation) are regular languages.
- No other languages over Σ are regular.

2.2.3 Regular Expressions

Regular expressions are a common way to describe a regular language in a practical way, it consists of constants, which denote sets of strings, and operator symbols, which then denote operations over these sets. The following definition is standard, and found as such in most textbooks on formal language theory. Given an alphabet Σ , regular expressions are built from the following components:

- $\emptyset$ and Σ are regular expressions.
- Every element in Σ is a regular expression.
- Concatenation of two regular expressions α and β , produces a regular expression $\alpha\beta$
- Union of two regular expressions α and β , produces a regular expression $\alpha \mid \beta$
- Kleene star of α is a regular expression.

The following is a recursive definition of the semantics of the regular expressions:

$$\begin{aligned}
 L(\emptyset) &= \emptyset, \\
 L(\varepsilon) &= \{\varepsilon\}, \\
 L(a) &= \{a\} \quad (a \in T), \\
 L(E + F) &= L(E) \cup L(F), \\
 L(EF) &= L(E)L(F), \\
 L(E^*) &= L(E)^*.
 \end{aligned}$$

For example, the regular expression $(ab)^* + c$ describes a language that consists of any number of repetitions of the string "ab" followed by the string "c".

$$\begin{aligned}
L((ab)^* + c) &= L((ab)^*) \cup L(c) \\
&= L(ab)^* \cup \{c\} \\
&= [L(a)L(b)]^* \cup \{c\} \\
&= [\{a\}\{b\}]^* \cup \{c\} \\
&= \{ab\}^* \cup \{c\}.
\end{aligned}$$

2.3 Finite Automata

Finite automata are among the most basic computational models in automata theory. They consist of a finite number of states and transitions between those states based on input symbols. They are capable of recognizing regular languages. A key concept in understanding how finite automata process input is the notion of a configuration. A configuration represents the current status of the automaton at a particular point during the computation. It is typically defined as a 2-tuple (q, w) , where q is the current state and w is the remaining input word. As the automaton processes the input, it transitions from one configuration to another based on the current symbol and the transition function. The input is said to be accepted if at least one of these paths reaches a final state with no remaining input. Any finite automaton can be made deterministic, and minimized. We will explore how minimization works later on in the FST section. While OFLAT uses a unified representation, two presentations are often discussed:

- **Deterministic Finite Automata (DFA):** For each state and input symbol, there is exactly one transition.
- **Nondeterministic Finite Automata (NFA):** Multiple transitions may be possible for the same input symbol in a given state.

An NFA may, in particular cases, behave deterministically.

2.3.1 Deterministic Finite Automata (DFAs)

Deterministic Finite Automata (DFA) are a special case of finite automata where, for each state and input symbol, there is exactly one transition to a next state. So there are no ε -transitions and no state has more than one transition for each input symbol. This determinism simplifies the computation process, as there is no ambiguity in state transitions. A deterministic finite automaton is formally defined as a 5-tuple [8]:

$$M = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q is a finite set of states,

- Σ is the input alphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function,
- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is the set of final (accepting) states.

The DFA operates as follows: it begins in the initial state q_0 , with the input mechanism positioned at the leftmost symbol of the input string. At each computational step, the automaton reads one input symbol, transitions to a new state as defined by δ , and advances the input mechanism one position to the right. This process continues until the entire input string has been consumed. If, at the end of the input, the automaton is in a state belonging to F , the input is accepted; otherwise, it is rejected.

The transition function δ dictates state changes. For example, if

$$\delta(q_0, a) = q_1,$$

then when the automaton is in state q_0 and reads the symbol a , it transitions to state q_1 .

Consider the DFA defined by:

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\}),$$

where the transition function δ is represented in the transition graph shown in Figure 1.2.

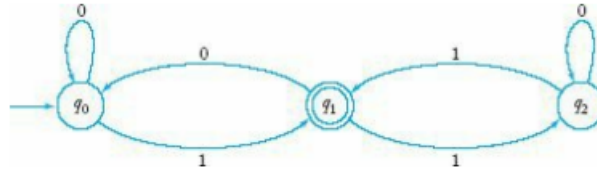


Figure 2.1: Example of a basic DFA.

This DFA accepts the string 01. Starting in state q_0 , the first symbol, 0, is read, and the automaton remains in state q_0 . Upon reading the next symbol, 1, it transitions to state q_1 . Since the input string has been fully consumed and the automaton is now in the final state q_1 , the string is accepted. Conversely, the string 00 is not accepted. After processing two consecutive 0's, the machine remains in state q_0 , which is not a final state. By similar reasoning, the automaton accepts strings such as 101, 0111, and 11001, but not 100 or 1100.

2.3.2 Nondeterministic Finite Automata (NFAs)

A Nondeterministic Finite Automaton (NFA) is the general case of finite automata, where there are no restrictions on the amount of transitions with the same input symbol from the same state, as well as ε -transitions. It is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q is a finite set of states,
- Σ is the input alphabet,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of final states,
- δ is the transition relation, a finite subset of

$$(Q \times (\Sigma \cup \{\epsilon\})) \times Q.$$

Essentially, each element of δ contains a pair consisting of a state and an input symbol (or ϵ), along with the corresponding next state. In a nondeterministic automaton, the value of the transition function is not a single element of Q but a subset of it. It defines the set of all possible states that can be reached by the transition.

For example, if the current state is q_1 , the input symbol a is read, and

$$\delta(q_1, a) = \{q_0, q_2\},$$

then either q_0 or q_2 could be the next state of the NFA.

ϵ (or λ) are accepted as the second argument of δ , meaning the NFA can make transitions without consuming an input symbol. It is also possible for it to remain in the same state during a ϵ -transition. Also, in an NFA, the set $\delta(q_i, a)$ can be empty, indicating that there is no transition defined for that specific combination of state and input symbol. A string is accepted by an NFA if there exists some sequence of configurations that places the machine in a final state at the end of the string. However, a string is rejected if there is no sequence of moves that can reach a final state. So we can look at nondeterminism as a mechanism by which the machine can always choose the correct path to acceptance, if one exists. Consider the following example of the above described transition graph in Figure 1.1. It describes a nondeterministic automaton since there are two transitions labeled a out of q_0 .

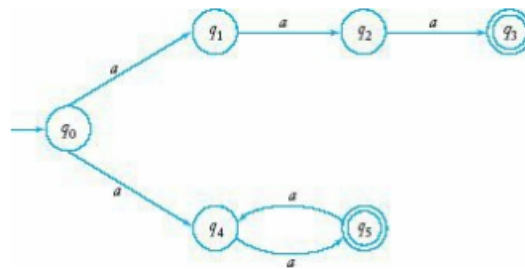


Figure 2.2: Example of a basic NFA.

2.4 Finite-State Transducers

Finite-State Transducers (FSTs) extend the concept of finite automata by associating output symbols with transitions, enabling the transformation of input strings into output strings. They are primarily classified as recognizers even though they generate an output. Formally, an FST is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q is a finite set of states; Σ the input alphabet; Γ the output alphabet; $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ the transition relation; $q_0 \in Q$ the initial state; and $F \subseteq Q$ the set of accept states. An FST processes a string from an input alphabet Σ and produces a string from an output alphabet Γ in response to the given input.

2.4.1 Operations on Finite-State Transducers

The following operations, which are also defined for finite automata, apply to finite-state transducers:[26]

- **Union** Given transducers T and S , there exists a transducer $T \cup S$ such that:

$$x[T \cup S]y \iff x[T]y \text{ or } x[S]y.$$

- **Intersection** Given transducers T and S , there exists a transducer $T \cap S$ such that:

$$x[T \cap S]y \iff x[T]y \text{ and } x[S]y.$$

- **Concatenation** Given transducers T and S , there exists a transducer $T \cdot S$ such that:

$$x[T \cdot S]y \iff \exists x_1, x_2, y_1, y_2 \text{ with } x = x_1x_2, y = y_1y_2, x_1[T]y_1 \text{ and } x_2[S]y_2.$$

- **Kleene Star** Given a transducer T , there might exist a transducer T^* such that:

$$(k1) \ \varepsilon[T^*]\varepsilon;$$

$$(k2) \text{ if } w[T^*]y \text{ and } x[T]z, \text{ then } wx[T^*]yz;$$

$$(k3) \text{ and } x[T^*]y \text{ holds only if implied by (k1) or (k2).}$$

The following operations are specific to finite-state transducers:

- **Composition** Given a transducer T on alphabets Σ and Γ and a transducer S on alphabets Γ and Δ , there exists a transducer $T \circ S$ on Σ and Δ such that:

$$x[T \circ S]z \iff \exists y \in \Gamma^* \text{ such that } x[T]y \text{ and } y[S]z.$$

- **Projection to an Automaton** There are two projection functions:

- π_1 preserves the input tape. Given a transducer T , there exists a finite automaton $\pi_1 T$ such that:

$$\pi_1 T \text{ accepts } x \iff \exists y \text{ such that } x[T]y.$$

– π_2 preserves the output tape, and is defined analogously.

- **Determinization** Given a transducer T , the goal is to construct an equivalent deterministic transducer—one with a unique initial state and no two transitions from any state sharing the same input label. The DFA determinization construction can be partially extended to transducers; however, output conflicts may prevent completion. Not all non-deterministic transducers admit equivalent deterministic versions (see Fig. 2.7). Characterizations of determinizable transducers have been proposed, namely the “twins property”: this property is a necessary and sufficient condition that characterizes determinizability. Note that there are efficient algorithms (albeit complex) for testing the twins property [1] and also efficient algorithms for determinization of transducers enjoying the “twins property” [11].
- **Minimization** There are several algorithms for minimizing transducers.

The two most commonly studied cases of FSTs are Mealy machines and Moore machines, which differ in how they produce the output based on the current state and input symbol. Later we will also see more about the general FSTs.

2.4.2 Mealy Machines

In a Mealy machine, the output produced by each transition depends on the current internal state and the input symbol used in the transition, so the output is generated during the transition itself.

A Mealy machine is defined as a 6-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, \theta, q_0),$$

where:

- Q is a finite set of internal states,
- Σ is the input alphabet,
- Γ is the output alphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function, which is a total function,
- $\theta : Q \times \Sigma \rightarrow \Gamma$ is the output function,
- $q_0 \in Q$ is the initial state of M .

The machine starts in the initial state q_0 , with the entire input available for processing. If at time t_n , the Mealy machine is in state q_i , the current input symbol is $a \in \Sigma$, and

$$\delta(q_i, a) = q_j, \quad \theta(q_i, a) = b,$$

then the machine will transition to state q_j and produce the output symbol $b \in \Gamma$. The process continues until the end of the input is reached.

Note that there are no final or accepting states associated with a transducer such as the Mealy machine, since the focus is on input-output transformations rather than language acceptance. The finite-state transducer (FST) is defined by the following components:

$$Q = \{q_0, q_1\}, \quad \Sigma = \{0, 1\}, \quad \Gamma = \{a, b, c\},$$

with initial state q_0 and

$$\begin{aligned} \delta(q_0, 0) &= q_1, & \delta(q_0, 1) &= q_0, \\ \delta(q_1, 0) &= q_0, & \delta(q_1, 1) &= q_1, \\ \theta(q_0, 0) &= a, & \theta(q_0, 1) &= c, \\ \theta(q_1, 0) &= b, & \theta(q_1, 1) &= a. \end{aligned}$$

This FST is represented by the graph shown in Figure 1.3. This Mealy machine prints the output string caab when given the input string 1010.

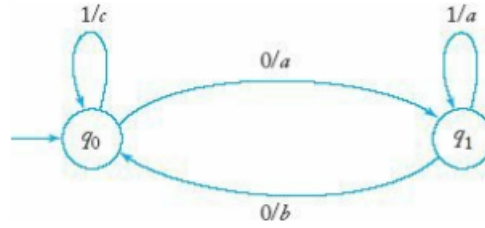


Figure 2.3: Example of a basic mealy machine.

2.4.3 Moore Machines

Moore machines differ from Mealy machines in the way output is produced. In a Moore machine, every state is associated with an element of the output alphabet. When the machine enters a given state, the output symbol is then produced. The output is generated only when a state transition occurs, so the symbol associated with the initial state is not printed at the start, but may be produced if the initial state is re-entered later.

A Moore machine is defined as a 6-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, \theta, q_0),$$

where:

- Q is a finite set of internal states,
- Σ is the input alphabet,
- Γ is the output alphabet,

- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function, which is a total function,
- $\theta : Q \rightarrow \Gamma$ is the output function,
- $q_0 \in Q$ is the initial state.

In the transition graph of a Moore machine, each state has two labels: its name and the associated output symbol.

Here we have a Moore machine M that takes strings of 0's and 1's as input. The output is a string of 0's until the first 1 occurs in the input, at which time it will switch to printing 1's. This continues until the next 1 occurs in the input, when the output reverts to 0. The alternation continues every time a 1 is encountered. For example, $FM(0010010) = 0011100$. This FST is a simple model for a flip-flop circuit.

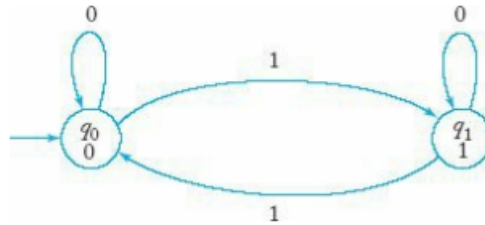


Figure 2.4: Example of a basic Moore machine.

2.4.4 Moore and Mealy Machine Equivalence

The examples in the previous sections showcase the differences between Moore and Mealy machines, but more importantly it allows us to see that they can be used to implement the same functions. In this sense, the two types of transducers can be considered equivalent.

Two finite-state transducers M and N are said to be equivalent if they implement the same function.[8]

The conversion of a Moore machine to an equivalent Mealy machine is straightforward. The states of the two machines are the same, and the symbol that is to be printed by the Moore machine is assigned to the Mealy machine transition that leads to that state.

The conversion from a Mealy machine to an equivalent Moore machine is more involved, because the states of the Moore machine must carry two pieces of information: the internal state of the corresponding Mealy machine and the output symbol produced by the Mealy machine's transition to that state.

In the construction, we create, for each state q_i of the Mealy machine, $|\Gamma|$ states in the Moore machine, labeled q_i^a for each $a \in \Gamma$. The output function for the Moore machine states is defined as

$$\theta(q_i^a) = a.$$

When the Mealy machine transitions to state q_j and produces the output symbol a , the equivalent Moore machine transitions into state q_j^a , thereby producing the same output a .

The Mealy machine in Figure 1.5(a) and the Moore machine in Figure 1.5(b) are equivalent.

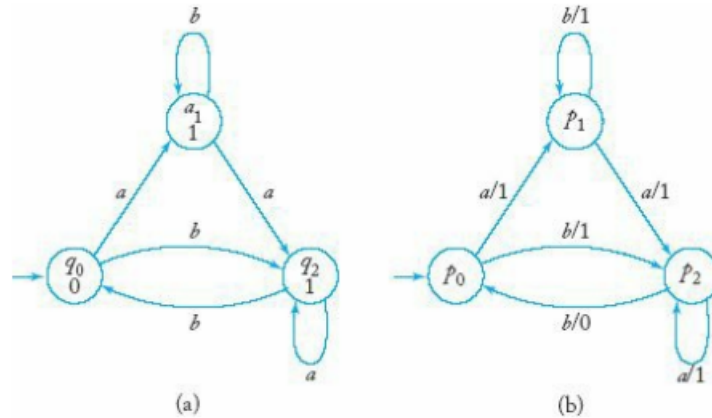


Figure 2.5: Example of a Mealy machine and its equivalent Moore machine.

2.4.5 Mealy Machine Minimization

For a given function $\Sigma^* \rightarrow \Gamma^*$, there are often many equivalent finite-state transducers, some of them differing in the number of internal states. For practical purposes, it is often important to find the minimal FST, that is, the machine with the smallest number of internal states.

The first step in minimizing a Mealy machine is to remove states that play no role in any computation because they cannot be reached from the initial state. When there are no inaccessible states, the FST is said to be connected. However, a Mealy machine can be connected and still not minimal, as the following example illustrates.

The Mealy machine shown in Figure 1.6(a) is connected, but it is clear that the states q_1 and q_2 serve the same purpose and can be combined to produce the machine in Figure 1.6(b).

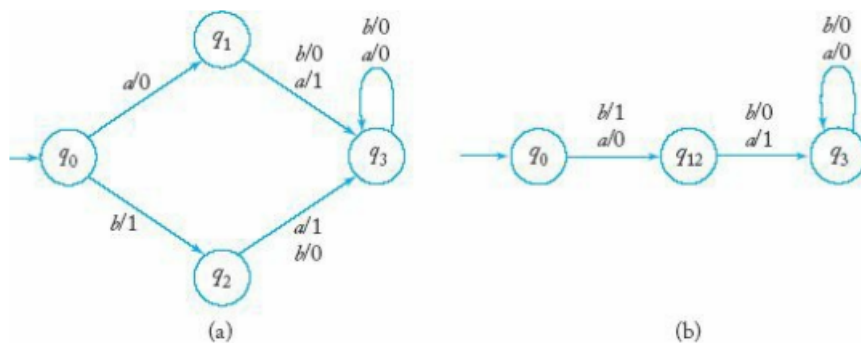


Figure 2.6: Example of a mealy machine and its minimal equivalent.

The minimization of a Mealy machine begins by identifying equivalent states—states that cannot be distinguished by any input string.[8]

Partition Algorithm

1. **Initial 1-equivalence partition:** Given the set of states $Q = \{q_0, q_1, \dots, q_n\}$, determine all states that are 1-equivalent to q_0 and partition Q into sets:

$$\{q_0, q_i, \dots, q_j\} \quad \text{and} \quad \{q_k, \dots, q_l\},$$

where the first set contains all states 1-equivalent to q_0 and the second those 1-distinguishable from it. Repeat this process with each state in Q , merging duplicates to form an initial partition based on 1-equivalence.

2. **Refinement:** For every pair of states q_i, q_j within the same equivalence class, examine their transitions. If there exist transitions on an input symbol leading to states in different equivalence classes, the current class to separate q_i and q_j . Check all pairs accordingly.
3. **Repeat:** Continue refining partitions until no further splits occur.

After the algorithm terminates, Q will be partitioned into equivalence classes $E_1, E_2, \dots, E_m$, with all states within each class being indistinguishable.

Constructing the Minimal Mealy Machine

A Mealy machine $M = (Q, \Sigma, \Gamma, \delta, \theta, q_0)$, we construct its minimal equivalent machine $P = (Q_P, \Sigma, \Gamma, \delta_P, \theta_P, E_P)$ as follows:

- Define the state set of P as the set of equivalence classes found in the partition:
 $Q_P = \{E_1, E_2, \dots, E_m\}$.
- For each class E_r and input symbol $a \in \Sigma$, select any state $q_i \in E_r$. If $\delta(q_i, a) = q_j$ and $\theta(q_i, a) = b$, define the transitions in P by

$$\delta_P(E_r, a) = E_s \quad \text{where } q_j \in E_s,$$

and

$$\theta_P(E_r, a) = b.$$

- The start state of P is the equivalence class E_P containing the original start state q_0 .

The resulting machine P is minimal and equivalent to M .

2.4.6 Moore Machine Minimization

The minimization of a Moore machine largely follows the same procedure used for Mealy machines, but with some differences due to the nature of Moore machines. In a Moore machine, outputs are associated with states rather than transitions. Therefore, states with different outputs are immediately 0-distinguishable (distinguishable without reading any input), and partitioning must start by separating states with different output symbols.[8]
 Moore machines: two states q_i and q_j are

- **0-distinguishable** if their output symbols differ;
- **k-distinguishable** if they were already $(k - 1)$ -distinguishable or if, for some input $a \in \Sigma$, their transitions lead to $(k - 1)$ -distinguishable states.

The partitioning algorithm proceeds as with Mealy machines: repeatedly refining equivalence classes until no further distinctions are possible. Once stable equivalence classes are found, the minimal Moore machine P is constructed by:

- defining states of P as the equivalence classes $E_1, E_2, \dots, E_m$,
- setting transitions so that if $\delta(q_i, a) = q_j$ with $q_i \in E_r$ and $q_j \in E_s$, then

$$\delta_P(E_r, a) = E_s,$$

and the output function by

$$\theta_P(E_s) = \theta(q_j),$$

using any representative q_j from the class E_s .

A complication arises in Moore machines however. The initial state's output may not be uniquely determined when the initial state cannot be re-entered, potentially resulting in minimal machines that are not identical under simple state relabeling. However, this issue is minor and does not affect correctness.

2.4.7 General Finite-State Transducers

The general case for finite-state transducers adds the possibility for non-determinism, including empty transitions, and final states.

Operations such as determinization, minimization[12], and output generation become more involved, with determination being impossible in certain cases.

Formally, a finite-state transducer T is a 6-tuple: [26]

$$T = (Q, \Sigma, \Gamma, I, F, \delta)$$

where:

- Q is a finite set, the set of **states**;
- Σ is a finite set, called the **input alphabet**;
- Γ is a finite set, called the **output alphabet**;
- $I \subseteq Q$ is the set of **initial states**;
- $F \subseteq Q$ is the set of **final states**;
- $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \times Q$ is the **transition relation**, where ε represents the empty string.

As we have a transition relation instead of a transition function, multiple transitions for the same input symbol from a single state are allowed, as well as ϵ -transitions. Furthermore there are also final states.

Determination of Non-deterministic FSTs

The determinization of finite-state transducers (FSTs) is an extension of the determinization algorithm used for finite automata. Each state in the resulting deterministic transducer corresponds to a set of original states. In this case however there is an added complexity because, unlike automata, transducers must handle both input recognition and output generation.

The result of the determinization process is typically a form of deterministic transducer that permits output strings to be associated with final states. When computation halts in such a state, the associated output is appended to the result. This design allows the transducer to emit outputs conditionally, depending on whether it has reached the end of the input string.

However, not all FSTs are determinizable, even if they define a function. In some cases, determinization fails because the decision on which path to take requires an unbounded lookahead. For example, in a given transducer, the system might need to read an unlimited number of symbols before choosing the correct transition. This leads to infinite branching in the determinization process, effectively causing the algorithm to loop endlessly without producing a result. Figure 2.7 is an example of a non-deterministic FST that cannot be determinized: [17]

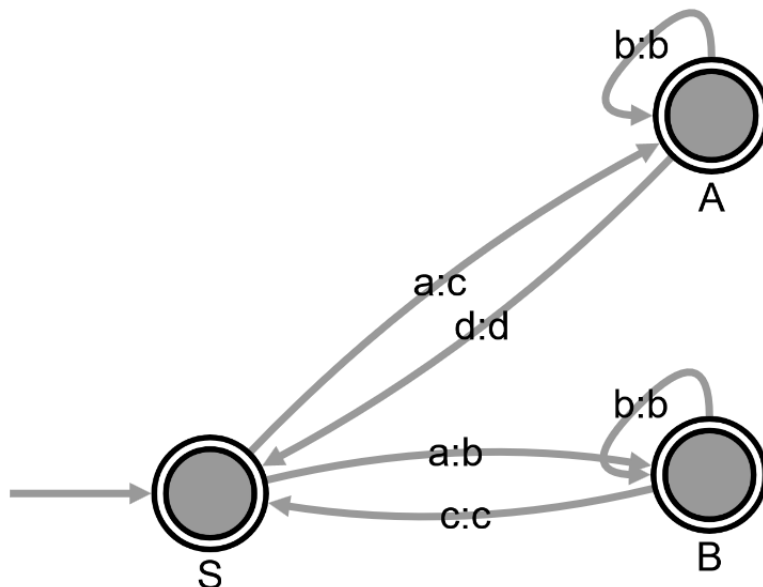


Figure 2.7: Example of a non-deterministic FST that cannot be determinized.

If the system is in state S and it reads an a , then it can only decide which transition

to take after reading an unbounded number of b 's. The algorithm enters an infinite loop for transducers for which no determinized version exists; in practice we detect and report non-termination.

2.4.8 Relevance to OCamlFLAT and OFLAT

The new improved OCamlFLAT library and the OFLAT web interface rely on FST concepts introduced in this chapter. A few practical connections are:

- **Output-aware acceptance:** features such as `acceptOut` and `acceptExpectedOutput` require tracking both the recognized input and the produced output, directly reflecting the FST configuration definition (q, w, o) .
- **Visualization of transitions:** the `inputEdges` facility used in OFLAT mirrors the transition relation δ for highlighting active paths during interactive execution.
- **Determinization and minimization:** the algorithms discussed here inform the implementation of deterministic FST checks and minimization routines exposed in the tooling.
- **Composition and projection:** OCamlFLAT leverages composition to chain multiple transducers and projection to obtain recognizers for either the input or the output side when needed.

2.5 Conclusion

Finite-State Transducers derive from foundational models of computation and extend the capabilities of finite automata by introducing output transformations. Their theoretical basis is related to regular languages (Type 3 of Chomsky's hierarchy). We also explored the two main types of FSTs, Mealy and Moore machines, which differ in how they produce output. Also explored their equivalence, minimization techniques. Also touched on the challenges of determinization and minimization of non-deterministic FSTs by looking at a general definition of FST. By understanding these concepts, it is now possible to delve into the specifics of FSTs and their applications, such as in the OFLAT application.

RELATED WORK

3.1 Introduction

In this chapter, we review existing pedagogical tools for teaching Formal Languages and Automata Theory (FLAT), with an emphasis on whether and how they support finite-state transducers (FSTs). We outline their key capabilities and usability aspects, then position OFLAT relative to them to contextualize the contributions of this work.

3.2 JFLAP: Java Formal Languages and Automata Package

JFLAP (Java Formal Languages and Automata Package)[22] is a widely adopted educational tool for teaching and learning formal languages and automata theory. Initially developed in the 1990s by Professor Susan H. Rodger and her students at Rensselaer Polytechnic Institute, JFLAP was first created in C++ and X Window under the name FLAP. It was later rewritten in Java and renamed JFLAP, enhancing its portability and usability. As a Java application, it runs on most systems equipped with the Java Runtime Environment (JRE).

JFLAP offers a comprehensive suite of features supporting the construction, simulation, testing, and conversion of theoretical models. It supports finite automata, formal grammar manipulation (regular, context-free, unrestricted grammars), Turing machines, and can convert between multiple of these models.

The tool provides immediate visual feedback by allowing users to simulate automata step by step, highlighting state transitions and input consumption. Figure 3.1 illustrates the model option in JFLAP.

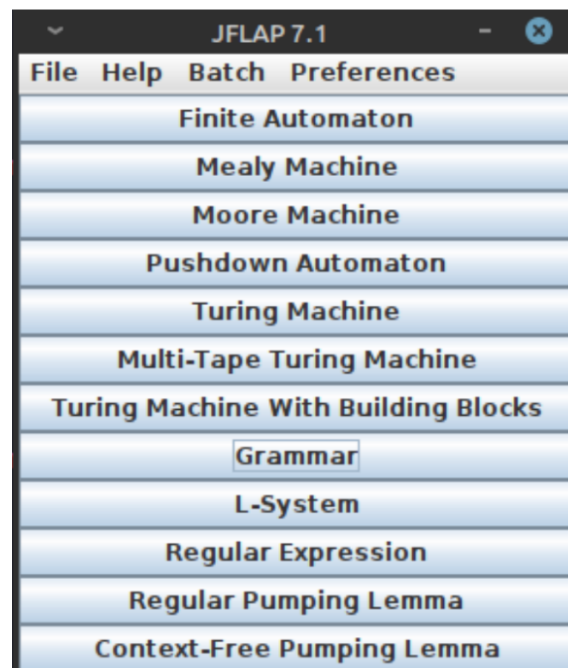


Figure 3.1: Model list available in JFLAP.

JFLAP also offers support for finite-state transducers in the form of Mealy and Moore machines. It has simulation and visualization, allowing users to observe state transitions alongside produced outputs. It does not support general transducers. Figure 3.2 shows a simple Mealy machine in JFLAP.

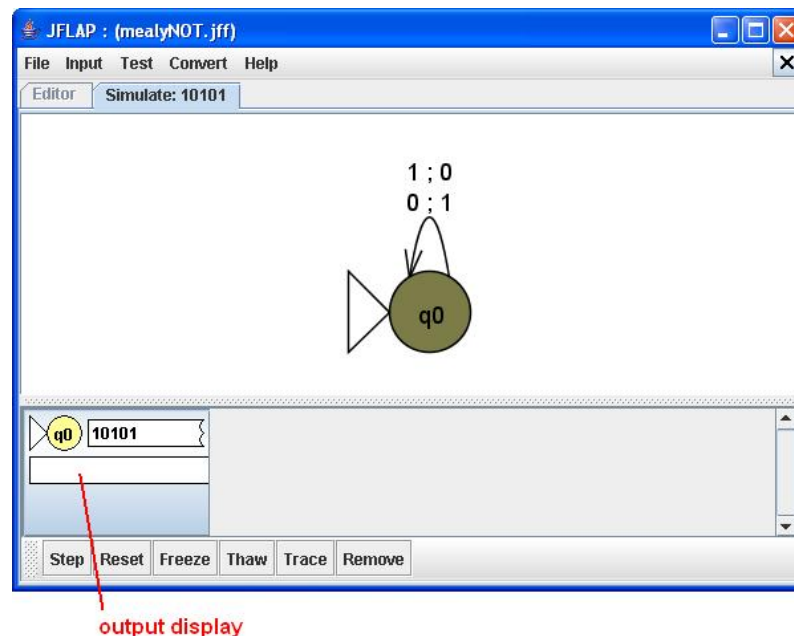


Figure 3.2: A simple Mealy machine in JFLAP.

This example shows us a Mealy machine includes a single state, q_0 , which serves as both the initial and active state throughout the computation. The transitions are represented

graphically by an arc, and are labeled in the form input ; output. In the example, we can see the transition with an input of 1 produces an output of 0, and vice versa. The input string 10101 provided below the state diagram, and the machine's output is displayed dynamically as the simulation progresses. The interface includes a control panel with options such as Step, Reset, Trace, and others, allowing users to simulate the machine step-by-step.

Conclusion

JFLAP distinguishes itself as one of the most complete and pedagogically effective tools in formal language education. It supports a wide variety of computational models and conversions grounded in formal theory.

3.3 FAdo

FAdo[21] is a Portuguese project developed at DCC/FCUP for automata, regular expressions, grammars, and transducers. It offers minimization, equivalence checking, classic language operations (union, intersection, complement), and conversions between automata and regular expressions. FAdo is code-first and does not ship a graphical or animated stepper;

FAdo is primarily intended to be used programmatically within a Python interpreter or in script-based workflows. As such, it does not provide a graphical user interface or native support for animated, step-by-step execution of automata. Instead, the FAdo library excels in flexibility and extensibility within a code-based environment. (e.g., DFA/NFA acceptance methods such as `evalWordP`/`evalWordNFA`, transition functions, epsilon-closure helpers).

For transducers, the `transducers` module(see [20]) provides deterministic and nondeterministic FSTs (DFT/NFT/GFT) and rational operations, making it a useful reference implementation for batch testing and cross-checking. The lack of a GUI makes it less immediate for beginners or visual walkthroughs, but its strength is flexibility within Python for programmatic construction and analysis.

3.4 Automaton Simulator

The Automaton Simulator[7] is an open-source, browser-based tool designed for constructing and simulating various types of automata. It supports deterministic finite automata (DFA), nondeterministic finite automata (NFA), and pushdown automata (PDA), which offers a simple and interactive environment for educational purposes. Although it does not support transducers, it is included here for its quality, simplicity, and breadth across classical automata.

The interface is divided into several functional sections that enhance usability. On the left-hand side Figure 3.3, users can input strings for testing. There are two main boxes:

one labeled *Accept* (one per line) for input strings expected to be accepted by the automaton, and one labeled *Reject* (one per line) for strings expected to be rejected. These strings can be tested in bulk using the *Bulk Testing* feature, which evaluates all entries at once and displays their acceptance or rejection. There is also a *Test/Debug* field, which allows for dynamic simulation.

The screenshot shows the left panel of the Automaton Simulator. At the top, there is a 'Test / Debug:' label above a text input field containing 'ABAB'. Below the input field are three icons: a green arrow pointing right, a clock, and a circular arrow. A horizontal line separates this section from the 'Bulk Testing' section below. The 'Bulk Testing' section has a label 'Bulk Testing' and a green arrow icon. Below this are two text areas. The first is labeled 'Accept (one per line):' and contains the text 'AB', 'ABAB', and 'ABABAB' on separate lines. The second is labeled 'Reject (one per line):' and contains the text 'A', 'B', 'ABA', 'BA', 'BB', and 'ABABB' on separate lines. Both text areas have a small diagonal line icon in the bottom right corner.

Figure 3.3: Left panel of the Automaton Simulator.

Below these input areas is the *Test Results* panel, where detailed feedback is shown after the test runs. Above the input section, buttons for loading, saving, and resetting the automaton are displayed. The central canvas of the interface is where the automaton is constructed Figure 3.4. States are created by clicking, and transitions are added by clicking and dragging between states. The simulation highlights the current state using a different color, the transition being taken, and the unprocessed portion of the input string. This is especially useful in step-by-step simulations where the learner can observe the exact sequence of transitions being executed.

Each state is displayed as a rounded rectangle, with different markings to indicate its role:

- A checked box denotes an accepting (final) state.
- Transitions are labeled with the input symbol that triggers them.

The simulator provides feedback on acceptance using textual cues such as "Accepted" or "Rejected", displayed near the top-left corner of the interface. During step-by-step execution, this feedback updates in real time as the input string is consumed.

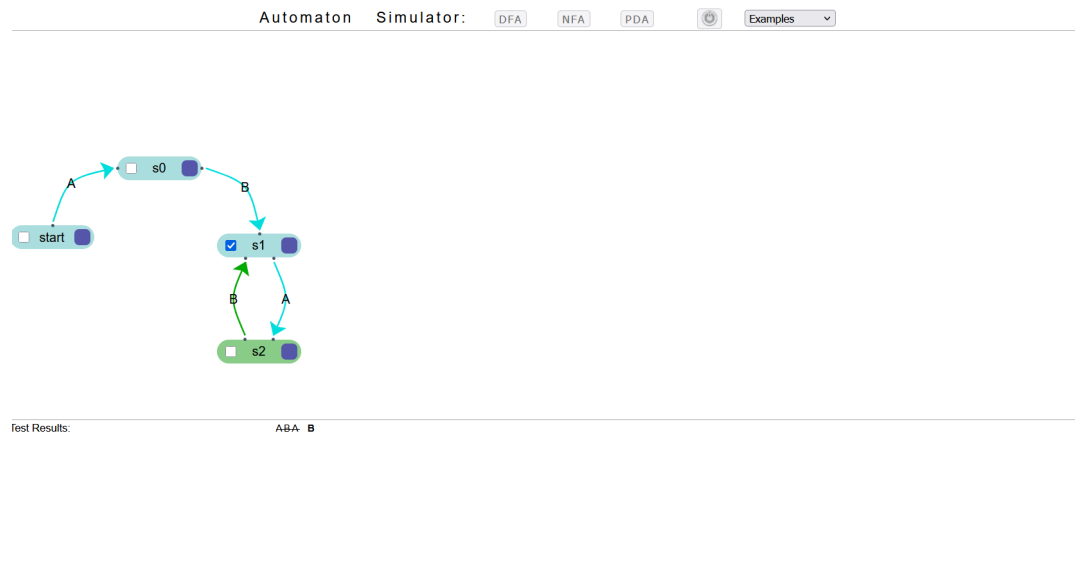


Figure 3.4: Central area of the Automaton Simulator.

Conclusion

The Automaton Simulator provides an effective and user-friendly platform for constructing and simulating automata directly in the web browser. Its strength lies in its simplicity and immediate feedback mechanism, which makes it a good tool for introductory coursework. Although it lacks the extensive model coverage and transformation tools found in more comprehensive platforms like JFLAP or OFLAT.

3.5 Flap.js

Flap.JS[5] is a web application therefore easily available on all kinds of devices, including mobile devices. It supports the graphical creation of Finite Automata, Regular Expressions, and Pushdown Automata, as well as tests and conversions, but none of these use animations, which would improve the learning process. In their web application, we can see that support for finite state transducers is not available, however, Mealy machines, Moore machines and turing machines are greyed out, indicating that they are planned features to be added in the future. Figure 3.5 shows the user interface, which while unusual can be a way to get perspective on different design approaches when it comes to graphical interfaces.

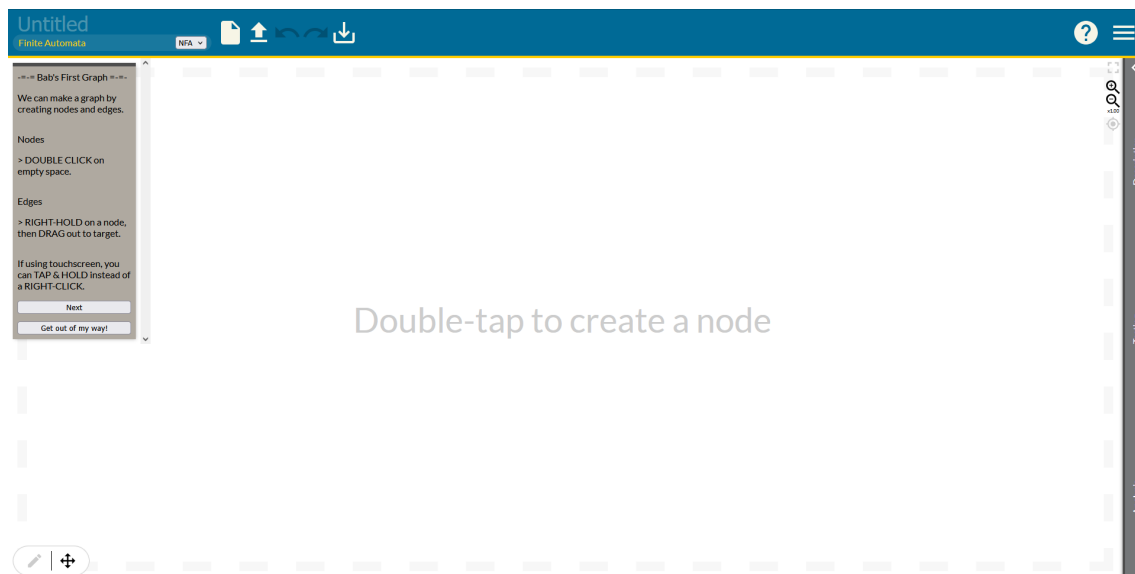


Figure 3.5: Flap.js user interface.

Conclusion

Flap.js, as it stands, does not provide as much variety as some of the others, like JFLAP and OFLAT. It also has a less intuitive interface than Automaton Simulator, which is simpler and provides similar functionalities.

3.6 Other tools

Additional tools referenced in the literature include TAGS [3], and other simulator efforts reported in academic theses [2]. Even when implementations are not readily available, they provide useful context on feature expectations for transducer-capable systems.

3.7 Comparison to OFLAT

Table 3.1 summarizes how the reviewed tools relate to OFLAT, focusing on model coverage, FST support, and pedagogical affordances.

Tool	Model coverage	FST support	Pedagogical/UX highlights
JFLAP	Broad (FA, PDA, TM, grammars)	Mealy/Moore only (no general FST)	Mature UI, conversions, step-by-step runs
FAdo (library)	FA/RE focus (code-based)	General FST (library)	Programmatic, no GUI; suited for scripting/batch tests
Automaton Simulator	FA, NFA, PDA	None	Simple browser UI, bulk testing, live highlighting
Flap.js	FA, RE, PDA (FST planned)	Planned	Web-based, unconventional UI, animations limited
OFLAT	FA, RE, PDA, FST	Yes (incl. Mealy/Moore and general FST)	Web UI with step/run, output-aware acceptance

Table 3.1: Positioning OFLAT relative to related educational tools.

Key differentiators for OFLAT:

- **Output-aware execution:** OFLAT exposes output checks (`acceptOut`, `acceptExpectedOutput`) aligned with FST semantics.
- **Interactive visualization:** transition highlighting and step/run modes mirror the underlying transition relation and outputs.

OCamlFLAT / OFLAT

This chapter presents the OCamlFLAT/OFLAT system as it stands currently. The system is built of two parts, the web application, which offers a graphical interface to the system, and the library it is built upon, which contains the logical part of the system. By understanding the OCamlFLAT/OFLAT system’s design and limitations, we can then have a better understanding of the new functionalities to be added.

4.1 Technology Stack

OCamlFLAT is implemented in OCaml. The OFLAT web application is generated from OCaml source using `js_of_ocaml`, with visualization powered by Cytoscape.js for graph rendering and vanilla JavaScript/HTML/CSS for UI composition. This stack keeps the domain logic and web UI in a single language ecosystem while leveraging established browser libraries for interactivity.

4.2 OCamlFLAT Library

The OCamlFLAT library is structured to provide a clear and modular framework for working with formal languages and automata. Its hierarchical module design starts with abstract concepts at the top, simplifying code organization and future expansions. At the top is the **Entity** module, which generalizes all other modules. The **Exercise** module supports exercise creation and validation, while the **Model** module offers shared utilities for all FLAT models, encouraging consistent implementations and code reuse. In this thesis, support for finite state transducers will be added to the library as well as all its necessary operations, which will be implemented as a new module. This includes output-aware acceptance, determinization/minimization routines where applicable, composition, and projections to automata. `TransducerSupport.ml` will define the core data types and supporting operations (including a parser for transducer specifications), and `Transducer.ml` will house the execution engine (the transducer simulator) and model-level algorithms.

Supporting modules handle essential tasks such as JSON serialization, error management, and foundational type definitions. Among these, the **BasicTypes** module plays a

key role by defining core constructs like symbols, words, states, and transitions—elements central to modeling automata and formal languages.

Each formal model in the library is represented by a pair of modules: one for auxiliary tasks like parsing and formatting, and another dedicated to core logic. The main logic modules include:

- **Regular Expression Module:** Implements word generation, word acceptance checking, conversion of regular expressions into finite automata, and simplification routines.
- **Finite Automaton Module:** Provides functions for creating, editing, minimizing, and analyzing finite automata, including conversions between deterministic and nondeterministic forms and detection of useful states.
- **Context Free Grammar Module:** Offers support for word generation and acceptance, validation, removal of left recursion and epsilon productions, and transformations to LL(1) grammars. It also provides computation of First and Follow sets, parsing table construction, and support for LR(0), SLR(1), LR(1), and LALR(1) parsing techniques.
- **Pushdown Automaton Module:** Extends FA functionalities to pushdown automata, covering acceptance, generation, and minimization, with mechanisms for validating determinism and identifying productive and reachable states.
- **Turing Machine Module:** Handles creation, modification, and analysis of Turing machines, including validation, determinism checking, linear boundedness checking, and conversions to finite automata where appropriate.
- **Composition Module:** Supports operations on model compositions, such as union, concatenation, intersection, and Kleene closure.
- **Poly Module:** Facilitates type conversions and interactions between different model types, including conversions across regular expressions, finite automata, context-free grammars, PDAs, and TMs.
- **Exercises Module:** Manages definition and manipulation of exercises, aligning with the system's pedagogical goals.

4.3 OFLAT

OFLAT is a web-based application providing an interactive interface built upon the OCamlFLAT library, with an emphasis on ease of use and educational effectiveness. Its design allows users to intuitively create, manipulate, and analyze a wide variety of formal language models.

Key features currently exposed in the UI include model creation/editing, loading examples, word acceptance and generation for automata and grammars, conversions (e.g., RE to FA), and stepwise tracing of runs.

4.3.1 User Interface Layout

OFLAT's interface is split into two primary sections:

1. **Side Menu:** Located on the left (shown in Figure 4.1), this menu lets users choose models such as finite automata, regular expressions, context-free grammars, push-down automata, and Turing machines. It also offers predefined examples for quick loading, options for word operations (acceptance, generation, tracing, conversions), and the ability to import or export models.
2. **Dynamic Workspace:** On the right side (shown in Figure 4.2), this area dynamically updates based on user interactions. It displays the current model visually and provides tools for editing, such as adding or removing states, transitions, or modifying grammar rules. The workspace often splits into a left panel for model structure and operations, and a right panel for operation results.

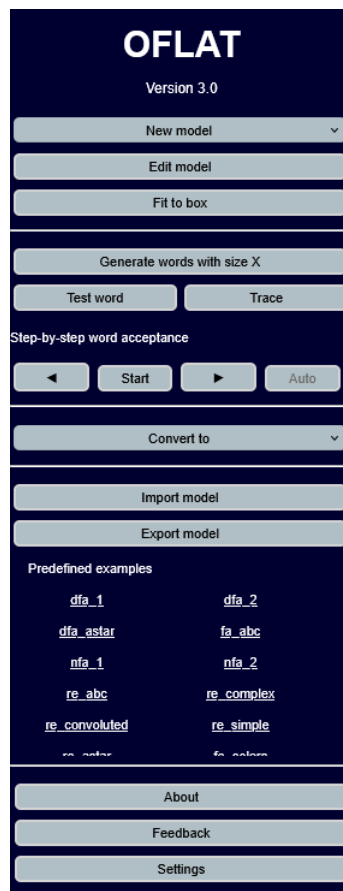


Figure 4.1: OFLAT Side Menu

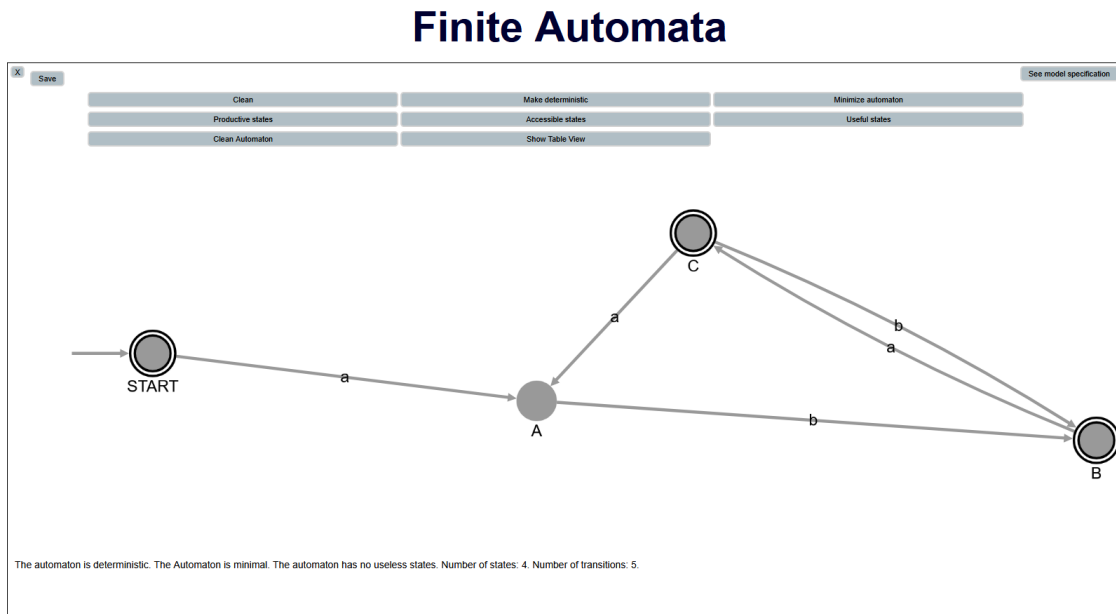


Figure 4.2: OFLAT Workspace showing a finite automaton

4.3.2 OFLAT Architecture

The OFLAT web application follows the Model-View-Controller (MVC) design pattern, which ensures a clear separation of responsibilities among its components:

- **Model:** The OCamlFLAT library acts as the model layer, handling all the logic and data structures related to formal language theory.
- **View:** This layer includes the graphical components of the system. The view modules are responsible for the visual representation of models and for handling user interaction.
- **Controller:** The controller modules process user input and turn them into commands for the view and model.

A standard operation in OFLAT typically follows this sequence:

1. The user interacts with the interface by providing input.
2. The controller receives the user command and, using the model (i.e., the OCamlFLAT library), performs the required computations.
3. The controller then updates the view to display the results.

This modular design enhances the system's maintainability and makes it easier to extend with new features or support for additional formal models in the future.

4.3.3 Current Limitations

- **FST coverage:** the existing UI and library APIs lack full support for finite-state transducers (acceptance with output conditions, determinization/minimization, composition).
- **Discoverability and guidance:** there is no integrated help section; button density can be overwhelming for newcomers.
- **Feedback depth:** stepwise execution highlights state transitions but does not yet surface output traces for FSTs.
- **Consistency:** some interactions differ slightly between model types (e.g., FA vs. PDA), increasing learning curve.

These gaps motivate the enhancements described in later chapters: extending the library with FST primitives and algorithms, exposing them in the OFLAT UI (including output-aware acceptance such as `acceptOut/acceptExpectedOutput`, and transition highlighting via `inputEdges`), and improving usability with clearer guidance.

4.4 Conclusion

Together, the OCamlFLAT library and the OFLAT web interface, provide a platform for learning and exploring formal language concepts through interactive, evolving tools. While the system has already achieved significant functionality, further enhancements are necessary to meet its full potential. For instance, the absence of a help section and the sometimes overwhelming arrangement of buttons in the interface can hinder user experience. Nevertheless, OFLAT supports future growth, allowing new models and functionalities to be added.

TECHNOLOGIES

This chapter provides an overview of the key technologies used in the system. We present the OCaml programming language, the mechanisms for interoperability between OCaml and JavaScript, and the use of additional libraries (such as Cytoscape.js) that enable both the functional and graphical aspects of the platform.

5.1 Why OCaml for OFLAT

OCaml offers strong static typing, pattern matching, and algebraic data types that map naturally to formal-language structures (symbols, words, transitions, grammars). Immutable defaults and exhaustive matches help prevent logic gaps in algorithms such as determinization, minimization, and composition of FSTs. The module system keeps model-specific code isolated yet composable, which suits the multi-model nature of OFLAT (FA, RE, CFG, PDA, TM, FST).

5.2 OCaml Programming Language

OCaml is a general-purpose, high-level, multi-paradigm programming language that extends the Caml dialect of ML with object-oriented features. It was created in 1996 by Xavier Leroy, Jérôme Vouillon, Damien Doligez, Didier Rémy, Ascánder Suárez, and others at the French Institute for Research in Computer Science and Automation (INRIA). The functional programming fragment of OCaml is based on the concept of expressions, where computations are performed by evaluating expressions that produce values, naturally supporting a declarative programming paradigm. [18]

5.2.1 Declarative Programming

Declarative programming focuses on writing code that solves problems by describing their logical properties rather than by elaborating an algorithm filled with operational details. In declarative programming[25], the emphasis is on specifying what the program should accomplish, rather than how to achieve it step by step.

For example, to determine the length of a list, a declarative solution might consist of a simple statement of the property: “the length of a list is one plus the length of its tail.” In contrast, an imperative solution would typically involve initializing a counter to zero and then traversing the list, incrementing the counter for each element. Declarative programming is often considered very high-level because it is closer to mathematical methods and human thought processes, as opposed to programming in a way that mimics machine operations.

Recursive functions written in a declarative style are always inductive functions, in which the problem is reduced to smaller instances of the same problem. This makes many complex problems easier to solve. The code also becomes more readable, as it resembles an equation and does not require mentally simulating its execution to understand it. When writing inductive functions, it greatly helps if the data themselves are inductive in nature—for example, using lists or trees. There are situations where writing a declarative solution still requires sophisticated considerations. For instance, to search for a value in an infinite binary tree, one must integrate the need to perform a breadth-first traversal into the solution. The exploration of infinite data structures is frequent in this project, as the configuration spaces that need to be explored are often infinite.

5.3 Interoperability using `js_of_ocaml`

One of the fundamental technologies enabling the OCamlFLAT/OFLAT system to function as a web-based tool is the `js_of_ocaml` framework. `js_of_ocaml`[\[19\]](#) is the interoperability solution used for bridging OCaml and JavaScript. Enabling the use of OCaml’s type system and functional programming features in web development. With this we can code the entire tool in OCaml. `js_of_ocaml` is not only a compiler but a collection of tools that includes a library, a binding mechanism, and a syntactic extension.

Compiler

At the core of `js_of_ocaml` is its compiler, which translates OCaml code into JavaScript code. This compilation step typically follows the standard OCaml compilation pipeline: the source code is first compiled using `ocamlc`, and the result is then passed to `js_of_ocaml` for translation into JavaScript. This allows us to have the type safety and early error detection provided by OCaml.

5.3.1 Bindings

Bindings are a crucial component for enabling interoperability between OCaml and JavaScript. In the `js_of_ocaml` ecosystem, bindings are used extensively to connect both languages, allowing OCaml code to access and manipulate JavaScript variables, functions and prototypes.

For example, the JavaScript `alert` function can be made available from OCaml using the following binding:

```
let alert (msg: string): unit = Js.Unsafe.global##alert (Js.string msg)
```

This snippet wraps the browser's `window.alert` in an OCaml function, converting the OCaml string to a JavaScript string before invocation.

As a second example, the following definition makes available on the OCaml side, the JavaScript constructor for the Cytoscape objects (that represents graphs).

```
let cytoscape_constructor: Js.Unsafe.t = Js.Unsafe.pure_js_expr "Cytoscape"
```

The following OCaml type, as shown below, describes the structure of the Cytoscape objects, and it allows method invocation with type safety:

```
class type cytoscape =  
  object  
    method add : DataItem.t t -> unit meth  
    method remove : DataItem.t t -> unit meth  
    method remove_fromSelector : js_string t -> unit meth  
    method mount : Dom_html.element t -> unit meth  
    method layout : layout_options t -> layout t meth  
    method resize : unit meth  
    (*...*)  
    method edges : js_string t -> DataItem.t Js.t js_array Js.t meth  
    method nodes : js_string t -> DataItem.t Js.t js_array Js.t meth  
    method tapdragover : DataItem.t t -> unit meth  
    method contextMenus : 'c t -> contextMenus Js.t meth  
    method popper : 'z -> popper Js.t meth  
  end
```

Syntax Extensions

To make JavaScript interoperation more idiomatic in OCaml, `js_of_ocaml` provides syntax extensions that support type-safe method invocation and property access. These extensions enable a natural coding style that mimics JavaScript syntax while remaining within OCaml's type system.

Library

Beyond the compiler and bindings, `js_of_ocaml` includes a library that exposes a substantial portion of the browser's API. This includes modules for:

- DOM manipulation (`Dom_html`, `Dom`)
- Event handling
- HTML5 APIs
- Canvas and graphics operations

5.4 Cytoscape.js for Graph Visualization

Cytoscape.js is an open-source JavaScript library for graph theory analysis and visualization [4]. In the context of the OFLAT system, it is used to render automata and tree structures graphically, providing users with an intuitive and interactive interface to construct and manipulate formal models.

To integrate Cytoscape.js with OCaml, the OFLAT system uses bindings defined in the `Cytoscape.ml` module. Graph operations such as adding nodes and edges are mapped to JavaScript via `js_of_ocaml`.

5.5 Web Technologies Used in OFLAT

Understanding OFLAT's web-based architecture requires familiarity with several core technologies:

- **Web browsers** act as clients that fetch and render web content, translating HTML, CSS, and JavaScript into interactive interfaces [14].
- **HTML** provides the structural markup for web pages, defining elements like headers, inputs, and buttons [15].
- **DOM** (Document Object Model) represents the HTML document as a tree of nodes that can be dynamically modified via JavaScript [13].
- **JavaScript** enables dynamic behavior and interaction within web pages, and is essential for manipulating the DOM [16].

OCamlFLAT LIBRARY IMPLEMENTATION

This chapter details the implementation of finite-state transducer (FST) support within the OCamlFLAT library. All logic described here resides primarily in two source files: `TransducerSupport.ml`, which defines the core data types, and `Transducer.ml`, which implements all algorithmic functionality. The implementation follows the same architectural conventions as the rest of OCamlFLAT, namely a clean separation between type definitions, validation, execution, and higher-level algorithms, each encapsulated in its own OCaml module.

6.1 Transducer Representation

The foundation of the FST representation in the library is the OCaml record type `t`, defined inside the `TransducerBasics` submodule of `TransducerSupport.ml`. This record captures every component of the formal definition of a finite-state transducer as a 6-tuple:

```
type t = {
  inAlphabet  : symbols;
  outAlphabet : symbols;
  states      : states;
  initialState: state;
  transitions : transitions4;
  acceptStates: states
}
```

The field `inAlphabet` holds the set of input symbols and `outAlphabet` holds the set of output symbols. These are kept as separate sets rather than unified, which makes it straightforward to validate that transition symbols are drawn from the correct alphabet and to enforce the semantic distinction between reading and writing. The `states`, `initialState`, and `acceptStates` fields mirror the corresponding components found in finite automata representations.

The most structurally significant type is `transitions4`, which is defined as a set of 4-tuples:

```
type transition4 = state * symbol * symbol * state
type transitions4 = transition4 set
```

Each element $(src, inputSym, outputSym, dst)$ represents a single transition: reading symbol $inputSym$ in state src causes the machine to emit $outputSym$ and move to state dst . Both $inputSym$ and $outputSym$ may take the value `epsilon` (represented as the string `~` in JSON and as the OCaml constant `epsilon` internally), permitting ϵ -transitions on the input side and silent transitions on the output side. This 4-tuple representation is the key distinction from the 3-tuple (src, sym, dst) used for finite automata, and it is consistently used across all modules that manipulate transducers.

The execution engine additionally defines a dedicated configuration type to track the full state of a computation at each step:

```
type configuration = state * word * word
type configurations = configuration set
type path          = configuration list
type trail          = configurations list
```

A configuration is a triple $(currentState, remainingInput, accumulatedOutput)$, representing the current control state, the portion of the input word not yet consumed, and the sequence of output symbols produced so far. A path is an ordered list of configurations tracing a single accepting run, and a trail collects the full set of configuration sets visited at each step of the computation, which is used to drive the step-by-step visualisation in the frontend.

Serialisation and deserialisation are handled by the `TransducerConversions` module. The `fromJSON` function reads a JSON object and constructs the OCaml record, using helper functions such as `JSON.fieldSymbolSet`, `JSON.fieldStateSet`, and `JSON.fieldQuadsSet` to parse each field. The corresponding `toJSON` and `toJSON2` functions produce JSON representations suitable for saving, loading, and transmitting the transducer definition. The `kind` field is set to the string `"transducer"`, which is the key used throughout the system to dispatch JSON objects to the correct model parser.

6.2 Validation and Properties

Before any algorithm is applied to a transducer, the `validate` function in `TransducerPrivate` performs a series of structural integrity checks. It verifies that neither alphabet contains `epsilon`, that the initial state is a member of the state set, that all accept states belong to the state set, and that all states and symbols appearing in transitions are drawn from their respective declared sets. Each failing check raises a descriptive error through the library's `Error` module. This validation is invoked automatically whenever a transducer is constructed via the `make` entry point.

Beyond basic structural validation, the implementation provides a rich set of property-checking functions that are exposed to the exercise system and to the frontend.

Determinism. The function `isDeterministic` checks whether the transducer is deterministic in the strict sense, meaning it has no ε -input transitions and each (state, input symbol) pair leads to at most one transition with at most one output symbol.

The function proceeds in two stages. First, it performs a global scan over all transitions to test for the presence of any ε -input transition, with the form `(_, epsilon, _, _)`:

```
let has_epsilon =
  Set.exists (fun (_, b, _, _) -> b = epsilon) fst.transitions
in
if has_epsilon then false
```

If any such transition exists the function immediately returns `false`, since ε -input transitions are inherently nondeterministic in this strict model: the machine can advance its state without consuming any input, which means that for the same input position there may be multiple reachable states.

If no ε -transitions are present, the function proceeds to the second stage: a nested iteration over every state and every input symbol in the declared input alphabet. For each pair `(st, input)`, it filters the full transition set to collect all transitions whose source state is `st` and whose input symbol is `input`. It then extracts two derived sets, the set of output symbols and the set of destination states and requires that all three quantities (number of matching transitions, number of distinct outputs, number of distinct destinations) are at most one:

```
Set.for_all (fun st ->
  Set.for_all (fun input ->
    let trs = Set.filter (fun (a,b,_,_) -> a = st && b = input) fst.transitions in
    let outs = Set.map (fun (_,_,out,_) -> out) trs in
    let dests = Set.map (fun (_,_,_,d) -> d) trs in
    Set.size trs <= 1 && Set.size outs <= 1 && Set.size dests <= 1
  ) fst.inAlphabet
) fst.states
```

The three conditions together enforce the full determinism requirement. `Set.size trs <= 1` ensures there is at most one applicable transition; `Set.size outs <= 1` ensures that transition has a unique output symbol; and `Set.size dests <= 1` ensures it leads to a unique next state. All three must hold simultaneously because the sets are derived independently and it is theoretically possible, due to the way transitions are stored, for the output set or destination set to have size greater than one even when the transition set has size one, if two transitions agree on source and input but differ on output or destination.

The overall result is true only if both stages pass for every state and every input symbol, confirming the transducer is fully deterministic with no ambiguity in either state or output.

A more permissive function, `isDeterministicEpsilon`, was attempted, which handles transducers that contain ε -transitions but may still be functionally deterministic.

Completeness. The function `isComplete` checks whether for every state and every non-epsilon input symbol there exists at least one outgoing transition. This is a standard completeness condition. It is used as a prerequisite for both the Moore and Mealy checks, since incomplete machines cannot be classical Mealy or Moore machines by definition.

Moore and Mealy Machines. The functions `isMooreMachine` and `isMealyMachine` verify membership in the two classical subclasses of FSTs.

For Moore machines, the key invariant is that the output symbol associated with a transition depends only on the source state, not on the input symbol. The implementation checks this by examining, for each state, the set of output symbols appearing on all outgoing transitions. If this set has cardinality at most one, the state's output is uniform and the Moore condition holds:

```
let state_has_single_output =  
  Set.for_all (fun st ->  
    let outgoing = Set.filter (fun (a,_,_,_) -> a = st) fst.transitions in  
    let outputs = Set.map (fun (_,_,c,_) -> c) outgoing in  
    Set.size outputs <= 1  
  ) fst.states  
in  
deterministic && complete && state_has_single_output
```

For Mealy machines, the condition is weaker: the output must be uniquely determined by the pair (state, input symbol), rather than by state alone. The implementation checks that for every (state, input) pair, the set of output symbols across all matching transitions has cardinality at most one. Both functions also require the machine to be deterministic and complete.

Reachability, Productivity, and Cleanliness. The library also implements state-level analyses mirroring those of finite automata. The function `reachable` computes the set of all states reachable from a given state via any sequence of transitions, using iterative forward exploration. The function `productive` identifies states from which some accepting state can be reached. The function `getUsefulStates` returns the intersection of reachable and productive states, and `cleanUselessStates` constructs an equivalent transducer containing only useful states and only the transitions between them, recomputing the alphabets from the surviving transitions. The function `isClean` delegates to the corresponding finite

automaton analysis by projecting the transducer to its underlying automaton structure using `asFiniteAutomaton`, which strips the output symbols from transitions.

6.3 Execution Engine

The execution engine is implemented in the `TransducerAccept` module and follows the same general framework used for finite automata and pushdown automata throughout OCamlFLAT. Computation proceeds by maintaining a set of active configurations and expanding them step by step through the `nextConfigs` function.

The initial configuration is constructed by `initialConfigs`, which wraps the start state, the full input word, and an empty output accumulator into a singleton set:

```
let initialConfigs (fst: t) (w: word) : configurations =
  Set.make [(fst.initialState, w, [])]
```

The `nextConfigs` function defines the single-step transition relation over configurations. Given a configuration (st, w, out) , it considers two cases. If the remaining input w is empty, only ϵ -input transitions from st are applicable, and each such transition $(st, \epsilon, outSym, st2)$ produces a new configuration $(st2, [], out @ [outSym])$ (with epsilon output symbols suppressed). If $w = x :: xs$, both symbol-consuming transitions on x and ϵ -input transitions on the current state are applicable, and the results are merged:

```
| x::xs ->
  let consume = Set.filter (fun (st1,sy,_,_) -> st1 = st && sy = x) fst.transitions in
  let viaConsume = Set.map (build_next_config xs) consume in
  let epsTr = Set.filter (fun (st1,sy,_,_) -> st1 = st && sy = epsilon)
    fst.transitions in
  let viaEps = Set.map (build_next_config w) epsTr in
  Set.union viaConsume viaEps
```

The helper `build_next_config` constructs the next configuration given the new remaining input and the matched transition, appending the output symbol to the accumulator when it is not epsilon.

A configuration is accepting when the control state is in `acceptStates` and the remaining input word is empty. The plain accept function simply tests whether any configuration in the reachable set eventually becomes accepting, delegating to the generic `Model.accept` combinator.

The function `acceptFull` returns a triple $(accepted, path, trail)$, where `path` is the sequence of configurations along one accepting run and `trail` is the full history of all configuration sets visited at each step. The trail is what the frontend uses to drive animated, step-by-step execution: each element of the trail corresponds to one round of

expansion, and the frontend renders the current state, remaining input, and accumulated output at each step.

The function `acceptOut` is a convenience wrapper over `acceptFull` that extracts only the output word from the final configuration of an accepting run, returning a pair `(true, outputWord)` on acceptance or `(false, [])` on rejection. This is the function used to answer the question “what does this transducer produce on this input?”, as opposed to the binary question of acceptance.

Two additional helpers enforce output validation for exercises. `acceptCheckOutput` first checks that the input belongs to the declared input alphabet, runs `acceptOut`, and then verifies that the produced output word belongs to the declared output alphabet. `acceptExpectedOutput` extends this by supporting exercise words written as `input->output`: it parses the arrow, feeds the input part to the transducer, and accepts only if the actual output exactly matches the expected one (and still belongs to the output alphabet). The exercise harness now uses `acceptExpectedOutput`, so authors can list cases such as `a->a`, `ab->ab`, etc., in the *inside* set to assert both acceptance and the precise output; any word without an arrow falls back to the alphabet-checked acceptance behaviour.

Output-aware acceptance helper:

```
let acceptExpectedOutput (fst: t) (w: word) : bool =
  let wstr = word2str w in
  match String.index_opt wstr '-' with
  | Some i when i + 1 < String.length wstr && wstr.[i + 1] = '>' ->
    let inp = String.sub wstr 0 i |> str2word in
    let out_exp = String.sub wstr (i + 2) (String.length wstr - i - 2) |> str2word in
    let (ok, out_act) = acceptOut fst inp in
    ok && out_act = out_exp && Model.checkWord fst.outAlphabet out_act
  | _ -> acceptCheckOutput fst w
```

This helper matches the library implementation: parse the arrow form, run the transducer on the input, and only accept when the produced output matches exactly and belongs to the declared output alphabet.

The `TransducerGenerate` module provides word generation functionality. Its `nextConfigs2` function reverses the direction of exploration: rather than consuming an input word, it explores outgoing transitions from the current state freely, accumulating both the input symbols read and the output symbols emitted. The `generate` function uses this to enumerate all input words of a given length that the transducer accepts, relying on the generic `Model.generate` infrastructure.

6.4 Algorithms

6.4.1 Determinization

The determinization algorithm is one of the most complex components of the FST implementation.

The implementation choices made for this algorithm were made after research and implementation attempts, and were necessitated by fundamental system limitations within the OCamlFLAT/OFLAT architecture. Specifically, the system’s transition data structure enforces that an output symbol on a transition must be a single symbol rather than an arbitrary output string or sequence. Consequently, supporting general delayed-output determinization, where ε -transitions carry outputs and varying output lengths accumulate across subset paths would violate this single-symbol transition constraint and break UI rendering and table mappings. While it could have been done, it would require significant changes to parts of the library that didn’t pertain to me. So the decision to restrict determinization to not handling transducers with output-producing ε -transitions, was made, as it bypasses the mentioned constraints, ensuring predictable execution, and a reliable interface for the web application.

`toDeterministic`, handles all transducers except those with output-producing ε -transitions. It begins by rejecting the input if any ε -transition has a non-epsilon output symbol. If this check passes, it applies a standard subset construction used in finite automata, adapted for transducers. The algorithm maintains a worklist of sets of NFST states (each representing a single DFST state), starts from the ε -closure of the initial state, and iterates by computing for each unprocessed state set and each input symbol the set of states reachable via that symbol followed by ε -closure. If at any point two transitions with the same source state and input symbol are found to produce different output symbols, the algorithm raises an internal exception `DeterminizationFailed` and returns the original transducer unchanged. The state names in the resulting deterministic transducer are synthesised by the `fusedfstState` function, which concatenates the names of the constituent NFST states into a canonical string representation enclosed in curly braces.

6.4.2 Minimization

The `minimize` function reduces a deterministic transducer to its minimal equivalent using a partition-refinement algorithm analogous to Hopcroft’s algorithm for finite automata [6], but adapted to account for output symbols, following the approach described by Mohri [12].

The algorithm begins by removing all useless states via `cleanUselessStates`, then partitions the remaining states into two initial blocks: accepting states and non-accepting states. It then iteratively refines this partition. In each refinement step, every block is split according to the *signature* of its member states, where the signature of a state is the set of triples (`inputSymbol`, `outputSymbol`, `destinationBlock`) derived from its outgoing

transitions. Two states have the same signature if and only if they are behaviourally indistinguishable with respect to input consumption, output production, and the equivalence class of their successors. The refinement continues until no further splitting is possible:

```
let rec refine (partition: states Set.t) : states Set.t =  
  let refined = Set.flatMap  
    (fun block -> Set.make (refine_block partition block)) partition in  
  if Set.equals refined partition then partition  
  else refine refined
```

Once the fixed point is reached, a representative state is chosen from each equivalence class (the first element of the class, as ordered by the set representation), and all transitions are re-mapped to use representatives. The resulting transducer has one state per equivalence class and is guaranteed to be the unique minimal deterministic transducer recognising the same input-output relation.

The function `isMinimized` checks whether a deterministic transducer is already minimal by running `minimize` and comparing the number of states. If the count is unchanged, the transducer is already minimal.

6.4.3 Conversion to Turing Machine

The function `asTuringMachine` converts an FST into an equivalent 2-tape Turing machine, where tape 1 acts as a read-only input tape and tape 2 acts as a write-only output tape. This is used to demonstrate the relationship between transducers and more general models of computation.

The conversion works as follows. Each FST transition (q_1, a, b, q_2) where a is not epsilon becomes a TM transition that reads a from tape 1, reads the blank symbol from tape 2, moves to state q_2 , writes a back on tape 1 (keeping it read-only), writes b on tape 2, and advances both heads rightward. If the output symbol b is epsilon, the tape 2 head stays in place and the blank is preserved. For ϵ -input transitions, a separate set of TM transitions is generated for each possible symbol on tape 1, keeping the tape 1 head stationary while still writing to tape 2. Finally, a dedicated accepting state `q_accept_tm` is added, and each FST accepting state receives a transition to it when both tapes read the blank symbol, signalling that the input has been fully consumed and processing is complete.

6.5 Composition and Operations

The `TransducerComposition` module implements five structural operations on transducers: composition, union, intersection, inverse, and concatenation. All are implemented using worklist-based state-space exploration or simple structural transformations.

Composition. Given two transducers $T_1 : A \rightarrow B$ and $T_2 : B \rightarrow C$, their composition $T_1 \circ T_2 : A \rightarrow C$ is the transducer that feeds the output of T_1 directly into T_2 . The implementation uses a product-construction worklist over pairs of states (q_1, q_2) , one from each transducer. Three cases of synchronisation are handled:

- **Lock-step:** T_1 consumes a non-epsilon input symbol a and emits b ; T_2 simultaneously consumes b and emits c . The composed transition is $(q_1, q_2) \xrightarrow{a/c} (q_1', q_2')$.
- **T_1 epsilon-step:** T_1 takes an ε -input transition emitting b ; T_2 consumes b and emits c . The composed transition is labelled with ε input.
- **T_2 epsilon-step:** T_2 takes an ε -input transition independently; T_1 stays at its current state. The composed transition is a pure ε step.

State names in the composed transducer are formed by concatenating the names of the component states with an underscore separator via the `fuse_states` helper. A pair of states is accepting in the composition if and only if both component states are accepting.

Union. The union $T_1 \cup T_2$ is constructed by introducing a fresh initial state and adding two silent ε -transitions, one to the renamed initial state of T_1 and one to the renamed initial state of T_2 . The states and transitions of both machines are renamed with suffix `_1` and `_2` respectively to avoid name collisions, and their sets are merged. This is the standard nondeterministic union construction for automata, directly adapted to transducers.

Intersection. The intersection $T_1 \cap T_2$ is defined as the transducer whose accepted pairs (w, v) are exactly those accepted by both T_1 and T_2 . The implementation uses a product construction where a transition is added to the result only when both machines simultaneously make a transition with identical input and identical output. This is more restrictive than composition and captures the set-theoretic intersection of the input-output relations. Three synchronisation cases are handled: lock-step moves where both machines agree on input and output, and two cases for independent silent $(\varepsilon, \varepsilon)$ steps by either machine.

Inverse. The inverse T^{-1} of a transducer T is formed by swapping the roles of the input and output alphabets and swapping the input and output symbols in every transition. The implementation is a simple structural map over the transition set and a swap of `inAlphabet` and `outAlphabet`. If T maps input word w to output word v , then T^{-1} maps v to w .

Concatenation. The concatenation $T_1 \cdot T_2$ accepts a pair (w, v) if and only if there exist $w = w_1 w_2$ and $v = v_1 v_2$ such that T_1 accepts (w_1, v_1) and T_2 accepts (w_2, v_2) . The implementation renames both machines to avoid state collisions, connects every accepting state of T_1 to the initial state of T_2 via a silent $(\varepsilon, \varepsilon)$ transition, and sets the accepting states of the result to be those of T_2 alone. The initial state of the concatenation is that of T_1 .

6.6 Integration Points Exposed to OFLAT

The library surface made available to the OFLAT frontend aligns with the features highlighted earlier:

- **Acceptance with outputs:** `acceptOut` returns the produced output word; `acceptExpectedOutput` enforces equality against an expected word (parsed from `input->output`) and validates both alphabets. These power output-aware exercises and quick checks.
- **Stepwise visualization:** `acceptFull` returns both a sample accepting path and the full trail of configuration sets. The frontend consumes the trail to highlight active states/transitions (via `inputEdges`) and to display the evolving output during stepping.
- **Generation:** `generate` enumerates accepted input/output pairs of bounded length, reused by the generic generation UI and exercise scaffolding.
- **Transformations:** determinization/minimization, inverse, union/intersection/concatenation, and composition are exposed so OFLAT can chain models and present derived transducers directly to the user.
- **JSON interoperability:** `toJson`/`fromJson` keep a stable `kind = "transducer"` tag, ensuring saved files load correctly across the web client and the OCaml library.

These hooks ensure the theoretical and algorithmic support described above is consumable by the web UI without duplicating logic in JavaScript.

OFLAT DESIGN AND FEATURES

This chapter deepens the description of the existing **graphical** OFLAT application (web-based) with a focus on how finite-state transducer (FST) support was integrated. The goal is to show how users interact with FSTs without disturbing established workflows for other automata models. Implementation details appear in [chapter 8](#).

7.1 Design Goals

The interface is shaped by four principles:

- **Discoverable actions:** core tasks (add states, create transitions, run, step, inspect) stay on-canvas through direct manipulation and context menus.
- **Pedagogical feedback:** color, overlays, and tables make nondeterminism and acceptance paths explicit; configuration counts and traces match textbook views for FA and extend cleanly to FST (with output words).
- **Consistency:** the same gestures and menus operate for FA, PDA, TM, and now FST;
- **Low-friction:** edits apply in place with immediate visual updates; auxiliary panes (tables, traces) live adjacent to the canvas so users can alternate between graph and data quickly.

7.2 Workspace and Graph Interaction

The canvas (Cytoscape.js) remains the primary surface for model construction and inspection. For FSTs, edges carry *input:output* label:

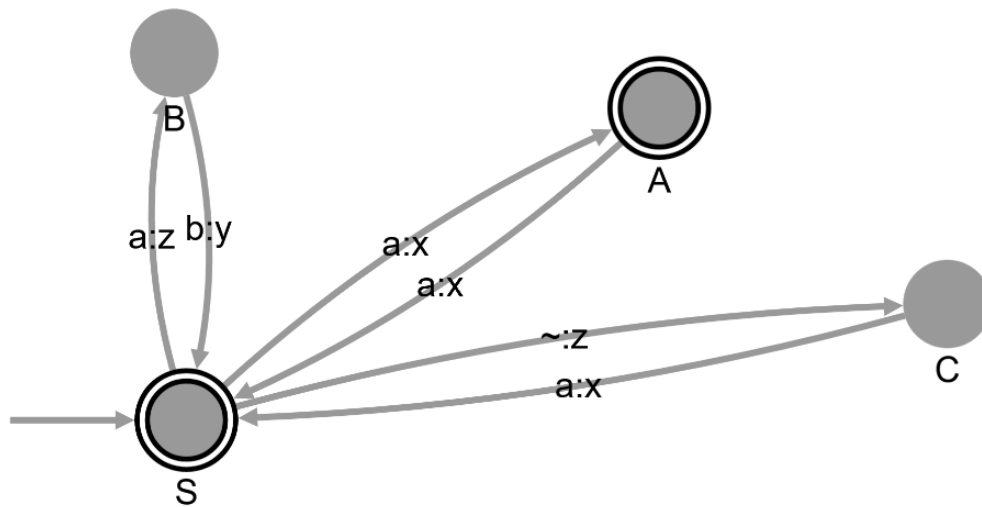


Figure 7.1: OFLAT canvas with input:output edge labels.

- direct add/move/rename of states; toggling initial/accepting via node menus;
- creation and removal of transitions with inline editors that accept both input and output symbols (epsilon allowed);
- zoom/pan with popper overlays that stay anchored to nodes even as the viewport moves.

Visual coding distinguishes initial and accepting states; a neutral base style is restored before each run so execution highlights remain readable.

7.3 Information Box, Poppers, and Context Menus

Node context menus expose per-configuration entries during step-by-step execution. Popovers display the count of active configurations on each node, giving quick feedback on nondeterministic branching. The information box surfaces the current configuration details: state, remaining input, and accumulated output (for FSTs). All overlays are lightweight and follow node movement to avoid obscuring the graph.

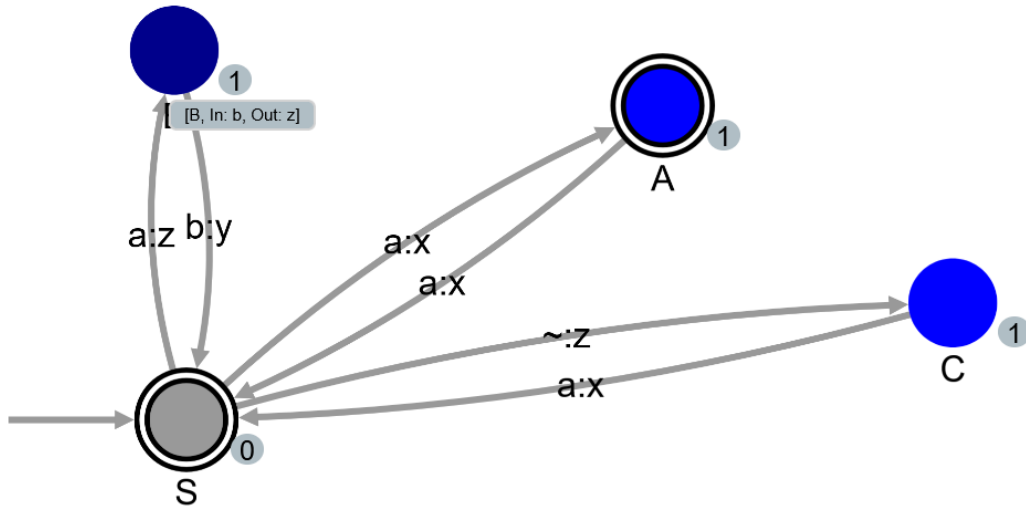


Figure 7.2: Information box and popper displaying state/input/output and configuration counts.

7.4 Step-by-Step Execution Experience

Execution highlights every active state at the current step. In nondeterministic scenarios, the path chosen by the engine is painted in a darker accent while alternative branches keep a lighter shade, making the selected witness path evident without hiding other possibilities. Users advance or rewind one step at a time; the input word view marks consumed vs. remaining symbols with a cursor separator. For FSTs, the accumulated output is shown alongside the remaining input.

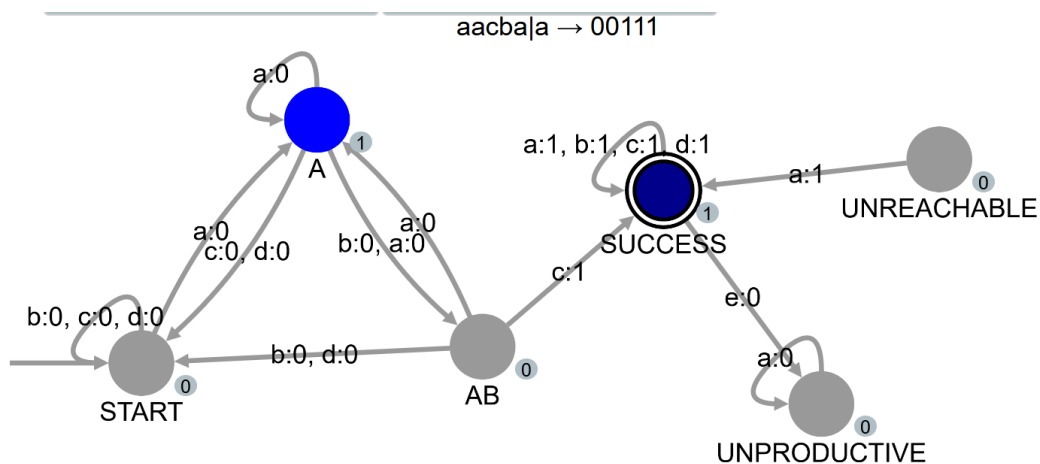


Figure 7.3: Step-by-step execution with highlighted path, input cursor, and accumulated output.

7.5 Trace Tables and Minimization Views

Tabular views mirror formal definitions: transition tables per input symbol (and input:output for FSTs), execution trace tables with state, remaining input, and produced output,

and minimization colorings that render equivalence classes on the graph. Tables rebuild automatically when the model changes, keeping visualization and data in sync.

Trace	Current State	Output	Word To Consume
0	START	~	aacbaa
1	A	0	acbaa
2	AB	00	cbaa
3	SUCCESS	001	baa
4	SUCCESS	0011	aa
5	SUCCESS	00111	a
6	SUCCESS	001111	~

Figure 7.4: Trace table for an FST run showing state, remaining input, and produced output.

7.6 Supported Models and Scope

The interaction model is uniform across finite automata, pushdown automata, Turing machines, and finite-state transducers. For FSTs, edge labels include input/output, configuration displays carry both remaining input and output, and popper counts reflect configuration multiplicity.

7.7 FST-Specific Workflows

The UI surfaces FST capabilities without diverging from existing gestures:

- **Create and edit transitions:** inline editors accept `input : output`, with epsilon allowed on either side; validation reuses the library alphabets to flag typos early.
- **Run vs. check outputs:** For exercises, `acceptExpectedOutput` lets inside sets specify `input->output` pairs, ensuring exact output matching.
- **Visualization hooks:** the configuration trail from `acceptFull` drives node highlighting and popper counts (via `inputEdges`); accumulated output is shown in the information box and trace tables at each step.
- **Transformations and minimization:** determinization/minimization (when applicable) and composition/union/intersection/inverse/concatenation are callable from the same menu used for FA/PDA/TM, returning a new FST rendered on the canvas.

- **Generation:** bounded word generation lists input/output pairs, mirroring the FA/PDA generation UI but including produced outputs.

7.8 Navigation to the Implementation View

Implementation details, module structure, event wiring, rendering logic, and the bridge from the library to the UI, are covered in [chapter 8](#).

OFLAT IMPLEMENTATION

This chapter describes how finite-state transducer (FST) functionality is integrated into the existing OFLAT code. It covers module responsibilities, rendering logic, and how the new pieces reuse pre-existing abstractions for other automata models. The goal is to provide FST support without altering the architecture established for FA, PDA, and TM. The design perspective is covered separately in [chapter 7](#).

8.1 Workspace and Graph Interaction

OFLAT keeps a shared rendering layer around Cytoscape.js, with thin OCaml bindings in `Cytoscape.ml`. FST support reuses the same primitives as finite automata:

- **Node lifecycle:** methods such as `addNode`, `addInitialNode`, and `addFinalNode` extend `TransducerView.model` (which inherits `AutomatonView.model`). They update the in-memory representation and immediately reflect changes on the graph (initial/accepting glyphs, labels).
- **Edges with input and output:** `inputEdges` maps each transition (`src, inSym, outSym, dst`) to a Cytoscape edge labeled `in:out`. Epsilon is rendered with the same tilde used elsewhere in OFLAT to stay consistent with existing models.
- **Style reset and repaint:** before runs, styles are reset (`Cytoscape.faStyle`) so later execution highlights remain visible. Minimization coloring and best-path accents reuse the same paint helpers used for FA.

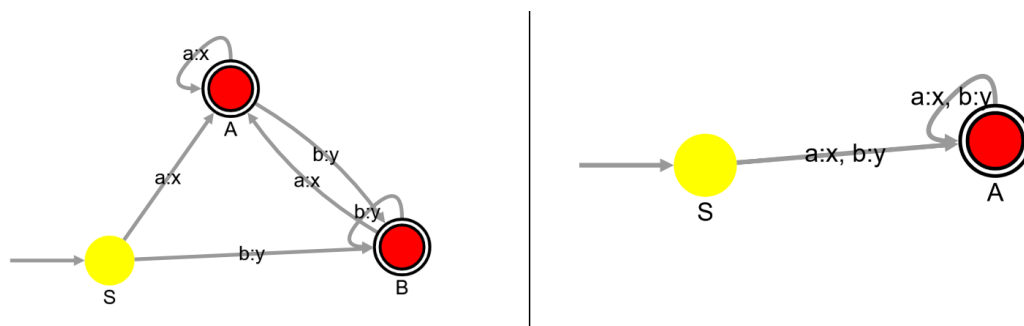


Figure 8.1: Rendering view with input:output edges and minimized coloring.

Edge construction for input/output labels:

```
method inputEdges (cy: Cytoscape.cytoscape Js_of_ocaml.Js.t) =
  let mapToCytoscapeEdge transitions =
    Set.map (fun (src, iSym, oSym, dst) ->
      let label = (symb2str iSym) ^ ":" ^ (symb2str oSym) in
      (src, label, dst)
    ) self#representation.transitions
  in
  Set.iter (Cytoscape.addEdge cy) (mapToCytoscapeEdge self#representation.transitions)
```

Each transition becomes a Cytoscape edge labeled `input:output`, using the same epsilon glyph as elsewhere.

8.2 Information Box, Poppers, and Menus

FST execution introduces richer configuration data (state, remaining input, accumulated output). The UI represents this via:

- **Configuration menus:** built with `contextMenus`, populated from configuration sets at each step. IDs are generated per-state to allow selective show/hide on hover. Menus are rebuilt whenever the step changes.
- **Poppers for configuration counts:** `buildPoppersConfigsCounter` creates poppers per node showing how many configurations currently live there. They track node movement and viewport changes via subscribed events (position, pan, zoom, resize).
- **Info display:** configuration details (state, remaining input, output so far) are presented in the info box so users can correlate graph highlights with exact words being processed.

8.3 Step-by-Step Execution Pipeline

Execution leverages the library `Transducer.acceptFull`, which returns (accepted flag, best path, trail of configuration sets). The view code keeps these in mutable fields and drives the UI as follows:

- **Trail and path storage:** `steps` holds the trail (array of configuration sets); `bestPath` stores a single accepting path. Statistics (visited configs, timing, exactness) come from `Model.stats()`.
- **Painting states:** each step paints all active states; accepting states use the same accent as FA. The chosen accepting path (if any) is overpainted in a darker color so users see which branch was selected among nondeterministic options.
- **Menus and poppers refresh:** on every step change, configuration menus are rebuilt and poppers refreshed to reflect the current configuration counts, ensuring hover interactions stay accurate.
- **Word and output view:** the sentence display marks consumed vs. remaining input with a bar and shows the accumulated output for the current configuration, aligning text feedback with graph highlights.

Initial and next steps:

```
method setInitialStep cy =  
  self#staticAcceptFull;  
  Cytoscape.resetStyle cy Cytoscape.faStyle;  
  if Array.length steps > 0 then self#initAllMenusAndFeatures cy steps.(0)  
  
method setNextStep cy =  
  if position < Array.length steps then  
    self#updateAllMenusAndFeatures cy steps.(position)
```

Step initialization runs the computation, resets styles, and builds menus/poppers; advancing a step refreshes the overlays using cached trail data.

UI data flow and state

The UI keeps a set of mutable fields in the model instance to avoid recomputation: (1) `steps` (the trail) and `bestPath`, (2) the current step index, (3) cached popper handles, and (4) the last rendered input/output strings. On a step change the view invalidates only what changed: it repaints nodes/edges, rebuilds menus and poppers from the cached trail slice, and reuses the existing Cytoscape elements instead of rebuilding the graph. Rendering throttles to the Cytoscape animation frame callback to avoid flicker on rapid scrubbing.

8.4 Tables, Traces, and Minimization

Tabular views rely on the shared `HTMLTable` helpers and adapt to FST specifics:

- **Transition tables:** built per input alphabet (plus epsilon if present). Cells aggregate destinations and output symbols as `dest/output`. Rebuilds occur after edits to keep data consistent with the graph.
- **Execution traces:** for a best path, the trace table lists step index, current state, produced output, and remaining input. The final row is colored green/red depending on acceptance.
- **Minimization coloring:** equivalence classes from the library minimization partition are painted on the graph using the same color palette as FA; edges are redrawn using `inputEdges` to keep labels intact.

8.5 Exercise Handling

Exercise evaluation for FSTs follows the same pipeline used for other models but is adapted to the transducer semantics:

- **Input $\rightarrow$ Output assertions:** Exercise words can be written as `input $\rightarrow$ output`. The view passes these strings to the library helper `acceptExpectedOutput`, which parses the arrow, runs the transducer on the input, and asserts the produced output matches exactly (and belongs to the declared output alphabet). Plain words without an arrow are handled by `acceptCheckOutput`, which enforces input and output alphabet membership.
- **Inside/Outside sets:** The inside set lists pairs the model must accept (e.g., `a $\rightarrow$ a`, `ab $\rightarrow$ ab`); the outside set can include counterexamples (e.g., `a $\rightarrow$ b`) to ensure incorrect mappings are rejected. The UI surfaces failures by highlighting the offending cases and showing the actual output returned by the transducer.
- **Properties:** Standard properties (deterministic, complete, Mealy/Moore, minimal, clean) are delegated to the library checks already described. Results are shown alongside input/output verdicts so users see structural issues and behavioural mismatches together.
- **Feedback loop:** On submission, the exercise runner collects acceptance results, output mismatches, and property violations, then updates the info box and tables. Because the same acceptance helpers drive both the exercise evaluator and the step-by-step view, traces shown in the UI match exactly what the checker has used.
- **UI specifics:** Failing inside cases are shown with the actual output alongside the expected arrow form; failing outside cases are flagged when the model accepts or

emits output where it should not. Property failures appear in the same panel so users see behavioural and structural issues together. The editor keeps the graph intact—only tables and status banners refresh—to reduce disruption while iterating on fixes.

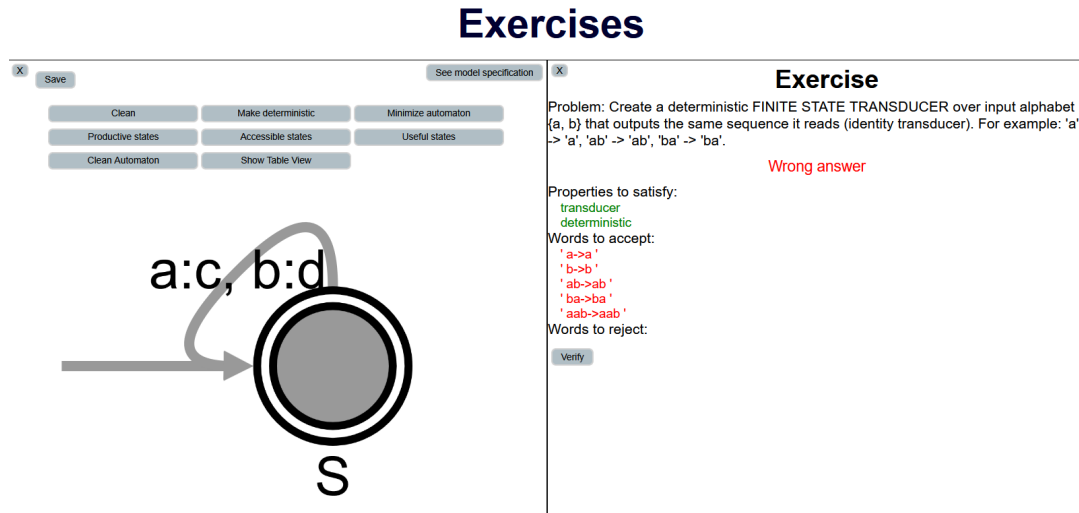


Figure 8.2: Exercise feedback: input→output cases with actual outputs and property results.

Exercise hooks exposed by the transducer module:

```
let checkExercise ex fst = Model.checkExercise ex
  (acceptExpectedOutput fst) (checkProperty fst)
let checkExerciseFailures ex fst = Model.checkExerciseFailures
  ex (acceptExpectedOutput fst) (checkProperty fst)
```

Exercises reuse the library's output-aware acceptance and property checks directly, so the UI surfaces the same verdicts as the core model.

8.6 Integration Boundaries

The FST view layer is intentionally thin: it delegates acceptance, generation, determinization, and minimization to library modules (`Transducer.accept`, `acceptFull`, `generate`, `minimize`), then renders the returned trail/partitions. This preserves OFLAT's existing separation between computation (library) and interaction (view). We also leverage the newer exercise-facing helpers `acceptCheckOutput` and `acceptExpectedOutput` to keep input/output validation consistent with the transducer library, ensuring UI traces and checks match the library's acceptance semantics. When determinization fails to terminate (e.g., output conflicts causing an infinite construction), the UI reports a non-termination message and leaves the existing graph intact rather than spinning indefinitely.

EVALUATION AND VALIDATION

This chapter summarizes how the finite-state transducer (FST) extensions were validated in both the OCamlFLAT library and the OFLAT application. It covers test layers, manual verification flows, and representative use cases that exercise some of the most relevant features.

9.1 Testing Methodology

Validation was performed in three layers:

- **Library unit tests (OCamlFLAT):** Tests live alongside model files (e.g., `TransducerTests.ml`) and target most functions such as `acceptOut`, `isDeterministic`, and `minimize`.
- **Integration checks (OFLAT):** The JS build produced by `js_of_ocaml` was tested in the browser: loading the app, creating/editing FST edges with input:output labels, running step-by-step execution, and verifying that the info box and poppers update with accumulated output and configuration counts. Exercises using the `input->output` format were validated to confirm UI feedback matches library behavior. Minimization and determinization actions were also triggered from the UI to ensure results matched library functions.
- **Manual exploratory runs:** For edge cases (epsilon outputs, nondeterministic branches, conflicting outputs, and invalid output symbols), transducers were crafted in the UI and run through the stepper to confirm highlighting, table rebuilds, and error surfacing behaved as described in Chapters 7 and 8.

Stress and scale checks. We exercised transducers with high branching factors to observe exponential blow-up in trails and UI responsiveness, and ran determinization on conflicting-output FSTs to ensure non-termination is detected and reported as “undefined/failed” rather than hanging the UI.

9.2 Use Cases

Each use case was exercised end-to-end: constructing the model in OFLAT, stepping through execution, and comparing results with the OCamlFLAT library output. This is not an exhaustive list of all features.

9.2.1 Case 1: Simulating a Moore Machine

A deterministic Moore machine was created with uniform outputs per state. The UI was used to:

- Draw the graph and label edges with input symbols (output fixed by state), then run the stepper to confirm the info box shows output determined solely by the current state.
- Run the exercise checker with inside examples (e.g., $a \rightarrow x$, $ab \rightarrow xy$) to verify `extttac` `ExpectedOutput` passes and that the trace table displays the expected output word.
- Invoke the property panel to confirm the `moore` and `deterministic` flags are satisfied.

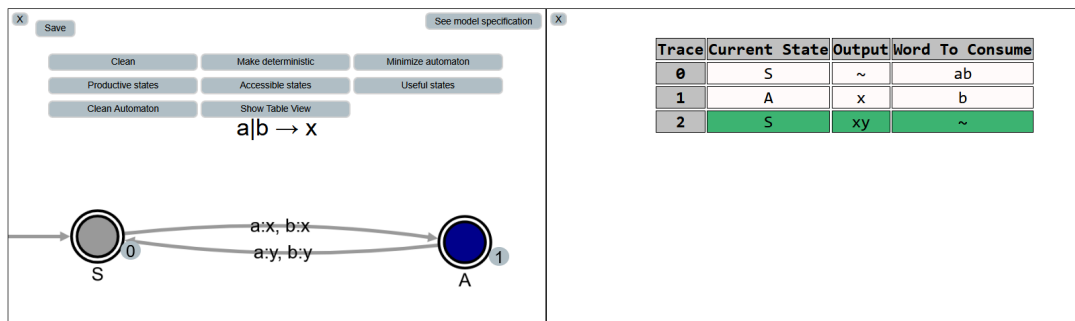


Figure 9.1: Moore machine execution highlighting state-based outputs and trace table.

9.2.2 Case 2: Non-Deterministic FST Execution

A nondeterministic FST with multiple outgoing edges per input was used to validate branching visuals and output tracking:

- Step-by-step execution highlighted all active states; the chosen accepting path was overpainted, while alternative branches stayed visible. The info box showed different accumulated outputs per branch.
- Poppers reflected configuration multiplicity; advancing a step recomputed counts without rebuilding the graph.

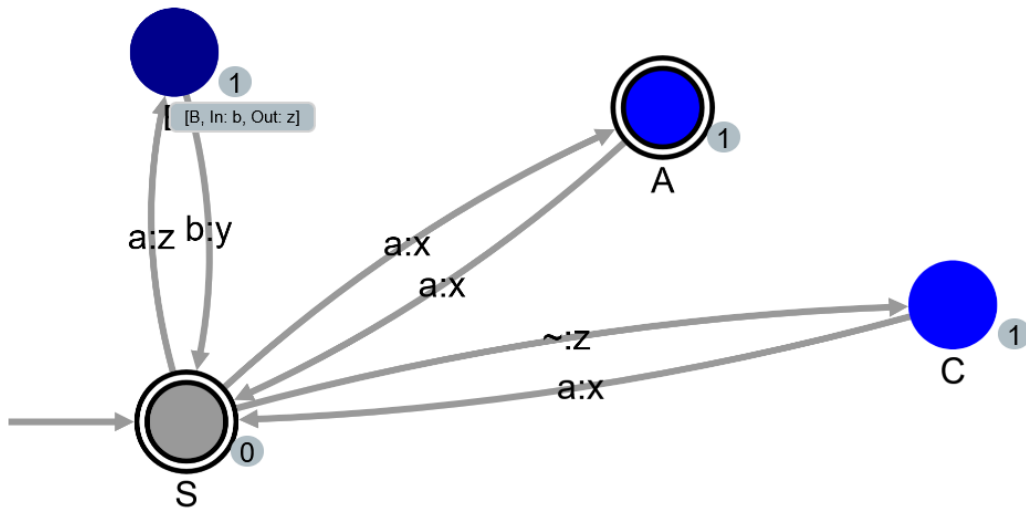


Figure 9.2: Nondeterministic FST execution with branch highlights, popper counts, and output tracking.

9.2.3 Case 3: FST Minimization

Minimization was tested on a deterministic FST with redundant states:

- The minimization routine produced a reduced graph; the UI recolored equivalence classes and redrew edges using `inputEdges` to preserve input:output labels.
- Execution before and after minimization yielded identical input/output behaviour (checked via `acceptOut`), and the trace table matched across versions.
- Property checks displays that both `deterministic` and `minimal` properties were satisfied after the minimization.

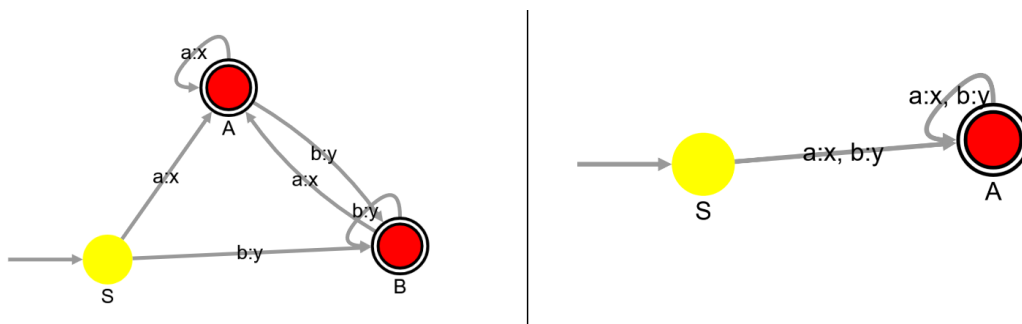


Figure 9.3: FST minimization view with equivalence-class coloring and preserved input:output labels.

9.2.4 Case 4: Determinization Failure (Conflicting Outputs)

An FST with conflicting outputs for the same (state, input), and one with epsilon transitions leading to nondeterminism was tested: Attempting determinization surfaced library errors, and the UI displayed the same message and preserved the original graph.

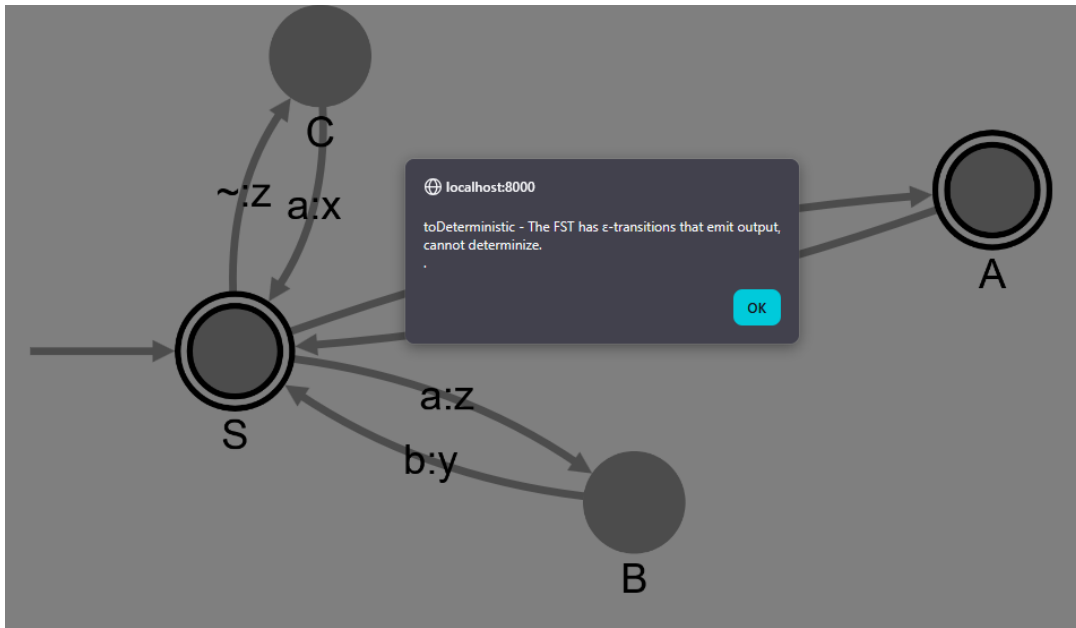


Figure 9.4: Determinization failure due to epsilon transitions; error surfaced while graph is preserved.

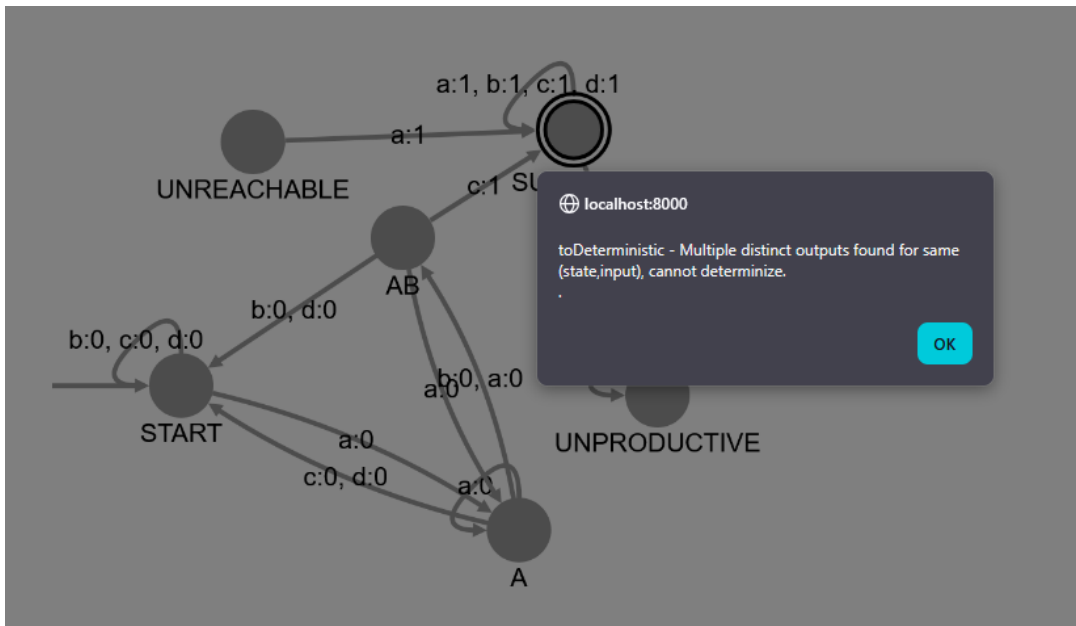


Figure 9.5: Determinization failure due to conflicting outputs; error surfaced while graph is preserved.

9.2.5 Case 5: Output-Aware Exercises

Exercises using the arrow notation were validated end to end:

- Inside examples like $ab \rightarrow xy$ passed only when the produced output matched exactly; outside examples like $ab \rightarrow xz$ were rejected and shown with the actual output in the feedback panel.

- The same cases run through the library's `acceptExpectedOutput` yielded identical verdicts, confirming consistency between UI and core logic.
- Invalid output symbols (not in the declared output alphabet) were flagged immediately, both in the UI and via `acceptCheckOutput`.

Exercises

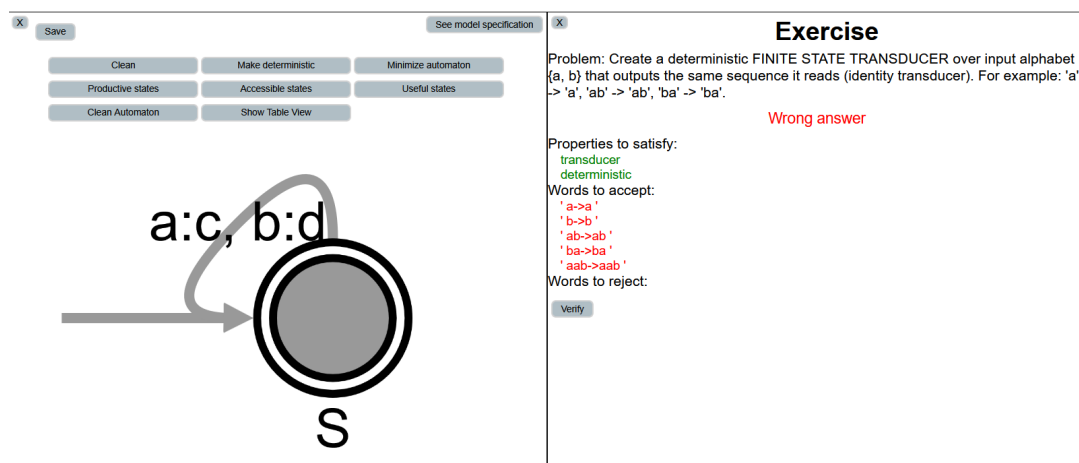


Figure 9.6: Output-aware exercise feedback with arrow notation, highlighting mismatches.

9.2.6 Case 6: Scale (Branching Explosion)

A highly branching nondeterministic FST was run to gauge responsiveness:

- Trail sizes grew exponentially with input length; the stepper remained responsive for short inputs but slowed as branches multiplied.
- Poppers and the info box updated correctly with large configuration sets.

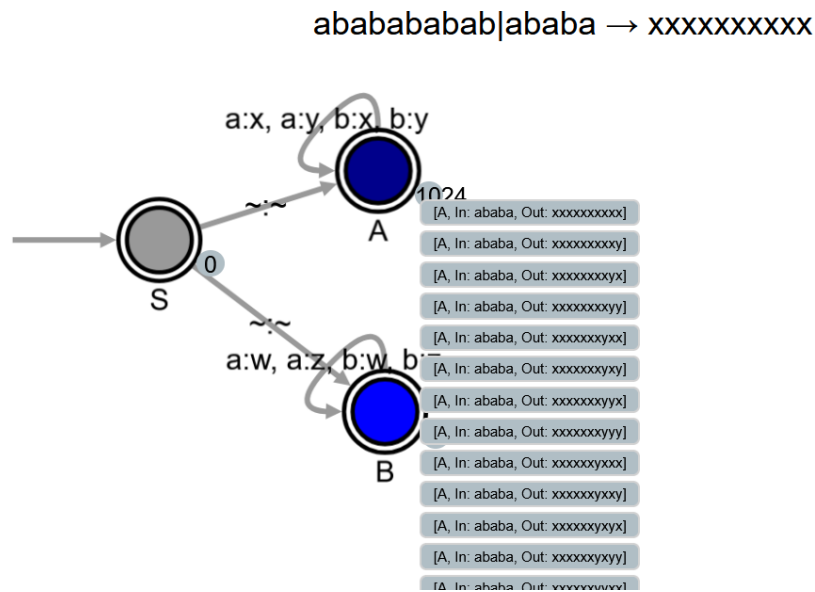


Figure 9.7: Scale test with high branching; UI remains responsive while showing many active configurations.

9.3 Discussion and Limitations

The graphical UI was not formally usability-tested; uniformity with existing models was prioritized, and a full UX study remains future work. Determinization non-termination is detected and reported as an undefined/failed result to avoid UI hangs. Performance degrades with exponential branching as expected; exploratory stress tests help bound practical input sizes.

CONCLUSION AND FUTURE WORK

10.1 Conclusion

This dissertation extended both the new improved OCamlFLAT library and the OFLAT web application with full support for finite-state transducers (FST). On the library side, we added output-aware acceptance functions (`acceptOut`, `acceptExpectedOutput`), minimization and determinism checks adapted to input/output labels, and utilities used by the UI (e.g., `inputEdges`). On the UI side, OFLAT now edits and executes FSTs with the same interaction patterns as other models: input:output edge labels, step-by-step execution with branch highlights, accumulated-output display in the info box, popper counts for nondeterminism, synchronized trace/transition tables, and exercise validation using the `input->output` format. The bridge built with `js_of_ocaml` keeps the browser client in sync with the OCaml models.

Evaluation combined unit tests in OCamlFLAT with manual and integration checks in OFLAT, exercising deterministic and nondeterministic runs, Moore-style outputs, and minimization. The result is a cohesive teaching tool where students can construct, visualize, step through, and assess FSTs with immediate feedback that matches formal definitions.

10.2 Summary of Contributions

- **Library:** New FST data model and algorithms (`acceptOut`/`acceptExpectedOutput`, determinization variants, minimization, composition/union/intersection/inverse/concatenation, projection, TM conversion) plus JSON interoperability for persistence.
- **UI:** End-to-end FST authoring and execution in OFLAT with input:output edge labels, trail-driven stepping, output-aware exercises, and reuse of Cytoscape-based visualization primitives.
- **Pedagogy:** Consistent interaction patterns across FA/PDA/TM/FST, output-aware checking for exercises, and tables/traces aligned with textbook definitions.

- **Tooling:** Single-language pipeline via `js_of_ocaml`, keeping logic and UI behavior synchronized.

10.3 Future Work

The work conducted in this dissertation will be integrated into the latest OFLAT release in the near future, this new version has various improvements and bug fixes on the UI side, but the modular approach of this implementation allows this integration to be seamless.

Besides that, several extensions could further improve the platform:

- **Exercise and grading flow:** support automatic generation of input/output test sets and richer feedback for counterexamples.
- **UX polish and accessibility:** Colorblind-safe palettes, and add lightweight tutorials or guided tours for first-time users.
- **Further models:** explore other transducer models (e.g., weighted, probabilistic) to expand the didactic scope while reusing the existing interaction patterns.
- **Performance and scale:** stress-test high-branching FSTs, add non-termination guards in determinization, and show clear guidance on practical limits.

BIBLIOGRAPHY

- [1] C. Allauzen and M. Mohri. “Efficient Algorithms for Testing the Twins Property”. In: *Journal of Automata, Languages and Combinatorics* 8.2 (2003), pp. 117–144. URL: <https://cs.nyu.edu/~mohri/pub/twins.pdf> (cit. on p. 11).
- [2] J. Andrade. *A Software Tool for Simulating Transducer Automata*. Master’s thesis. 2014. URL: <https://www.dcc.fc.up.pt/~nam/resources/docs/teseJA.pdf> (cit. on p. 24).
- [3] S. Chakraborty and D. Xin. *TAGS: A Software Tool for Simulating Transducer Automata*. Research report. 2011. URL: https://www.researchgate.net/publication/228577556_TAGS_A_Software_Tool_for_Simulating_Transducer_Automata (cit. on p. 24).
- [4] C. Contributors. *Cytoscape.js*. 2024. URL: <https://js.cytoscape.org/> (cit. on p. 34).
- [5] Flap.js Contributors. *Flap.js Web Application*. 2024. URL: <https://flapjs.github.io/FLAPJS-WebApp/> (cit. on p. 23).
- [6] J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd. Boston, MA, 2006. ISBN: 978-0-321-45536-9 (cit. on p. 41).
- [7] Kyle Dickerson. *Automaton Simulator*. 2024. URL: <https://automatonsimulator.com/> (cit. on p. 21).
- [8] P. Linz. *An Introduction to Formal Languages and Automata*. 5th. Jones & Bartlett Learning, 2011. ISBN: 978-0-7637-7964-8 (cit. on pp. 7, 13–15).
- [9] J. M. Lourenço. *The `aidisclose` package: Generative AI disclosure checklist and statements*. Version 1.11.0. 2025. URL: <https://ctan.org/pkg/aidisclose/aidisclose-doc.pdf> (cit. on p. vi).
- [10] J. M. Lourenço. *The NOVAtesis L^AT_EX Template User’s Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf> (cit. on p. iii).

- [11] M. Mohri. “Finite-State Transducers in Language and Speech Processing”. In: *Computational Linguistics* 23.2 (1997), pp. 269–311. URL: <https://aclanthology.org/J97-2003/> (cit. on p. 11).
- [12] M. Mohri. “Minimization Algorithms for Sequential Transducers”. In: *Theoretical Computer Science* 234 (1998). DOI: [10.1016/S0304-3975\(98\)00115-7](https://doi.org/10.1016/S0304-3975(98)00115-7) (cit. on pp. 16, 41).
- [13] M. D. Network. *Document Object Model (DOM)*. 2024. URL: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model (cit. on p. 34).
- [14] M. D. Network. *How Browsers Work*. 2024. URL: https://developer.mozilla.org/en-US/docs/Web/Performance/How_browsers_work (cit. on p. 34).
- [15] M. D. Network. *HTML - HyperText Markup Language*. 2024. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML> (cit. on p. 34).
- [16] M. D. Network. *JavaScript*. 2024. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (cit. on p. 34).
- [17] G. van Noord and D. Gerdemann. “Finite State Transducers: Algebraic Properties and Composition”. In: *Finite State Devices for Natural Language Processing*. Ed. by K. Templeton, L. Karttunen, and R. M. Kaplan. <https://www.let.rug.nl/vannoord/papers/fsa/node23.html>. Lecture Notes in Computer Science, Springer, 1997 (cit. on p. 17).
- [18] OCaml Team. *About OCaml*. 2024. URL: <https://ocaml.org/about> (cit. on p. 31).
- [19] Ocsigen Team. *js_of_ocaml Manual*. 2025 (cit. on p. 32).
- [20] F. Project. *FAdo Transducers Documentation*. 2025 (cit. on p. 21).
- [21] F. Project. *FAdo: Tools for Formal Languages Manipulation*. 2025 (cit. on p. 21).
- [22] S. H. Rodger. *JFLAP: Java Formal Languages and Automata Package* (cit. on p. 19).
- [23] The L^AT_EX Project team. *L^AT_EX – A document preparation system*. 1985. URL: <https://www.latex-project.org> (cit. on p. vi).
- [24] Wikipedia contributors. *Chomsky hierarchy — Wikipedia, The Free Encyclopedia*. 2024. URL: https://en.wikipedia.org/wiki/Chomsky_hierarchy (cit. on p. 5).
- [25] Wikipedia contributors. *Declarative Programming*. 2023 (cit. on p. 31).
- [26] Wikipedia contributors. *Finite-state transducer — Wikipedia, The Free Encyclopedia*. 2024. URL: https://en.wikipedia.org/wiki/Finite-state_transducer (cit. on pp. 10, 16).
- [27] Yana Suchikova and Natalia Tsybuliak and Jaime A. Teixeira da Silva and Serhii Nazarovets. “GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing”. In: *Accountability in Research* (2025), pp. 1–27. DOI: [10.1080/08989621.2025.2544331](https://doi.org/10.1080/08989621.2025.2544331) (cit. on p. vi).



2026

Finite-state transducers in OCamlFLAT/OFLAT

João Santos